Software Engineering

BCSE301L

Module 3

Dr. Naveenkumar Jayakumar
Dept. of Computational Intelligence
PRP-217-4

Software Requirements

- Descriptions of services a system should provide.
  - Constraints on system operation.
  - Reflect customer needs (e.g., controlling a device, placing an order).

- Software requirements are the descriptions of the services that a software system should provide and the constraints under which it must operate. They essentially define what the software should do to meet the needs of its users and stakeholders.

Software Requirements

- What the software should do: This encompasses the **functionalities and features** expected from the system. Examples include:
  - Processing user inputs
  - Storing and retrieving data
  - Generating reports
  - Controlling external devices

Software Requirements

- Constraints on operation: These are **limitations** or restrictions **on how the software should function**. Examples include:
  - Performance requirements (e.g., response time, throughput)
  - Security requirements (e.g., data encryption, access control)
  - Usability requirements (e.g., ease of use, accessibility)
  - Compatibility requirements (e.g., operating system compatibility)

Software Requirements - Types

Types of software requirements

Business requirements	User requirements	Software requirements
Outline measurable goals for the business.	Reflect specific user needs or expectations.	Identify features, functions, non-functional requirements and use cases.
Define the why behind a software project.	Describe the who of a software project.	Delve into the how of a software project.
Match project goals to stakeholder goals.	Highlight how users interact with it.	Describe software as functional modules and non-functional attributes.
Maintain a BRD with requirements, updates or changes.	Create a URS, or make them part of the BRD.	Compose an SRS, and, optionally, an FRS.

Software Requirements - Types

Business Requirements

- The high-level goals and objectives that a software system must achieve to support the organization's strategic business needs. They focus on the "why" behind the software and its impact on the business.

Functional Requirements

- Describe the specific functions that the software must perform

Non-Functional Requirements

- Describe the quality attributes of the software, such as performance, usability, and security.

User requirements

- Describe the needs and expectations of the end-users of the software

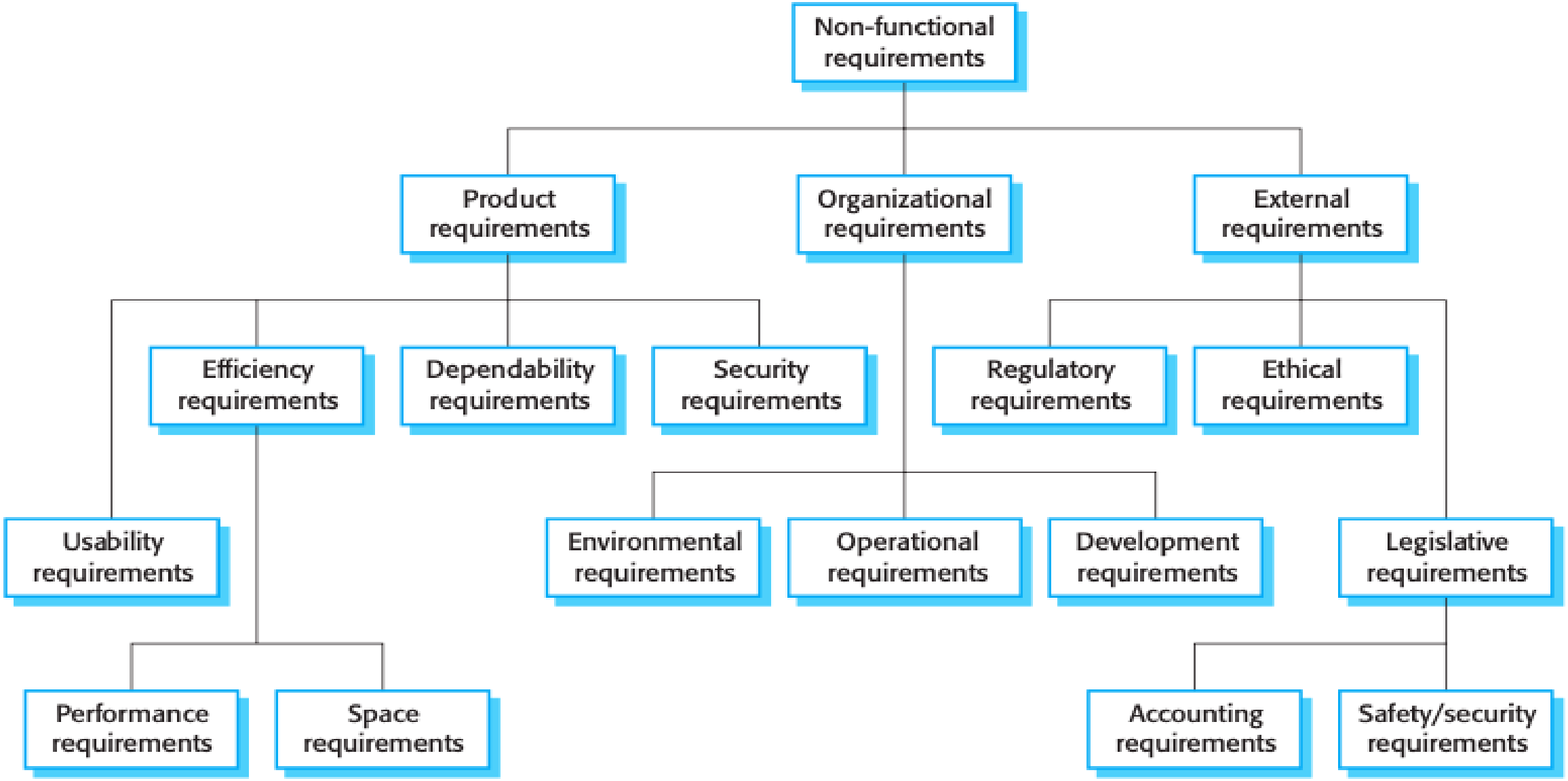
System requirements:

- Describe the overall requirements of the entire system, including the software and hardware components

Software Requirements - Types

	Functional requirements	Non-functional requirements
Objective	Defining what the product does	Describing how the product works
End result	Product features	Product properties
Focus	On user needs	On user expectations
Documentation	Software requirements specification, use cases, user stories, work breakdown structures, prototypes	Software Requirements Specification
Origin	Usually defined by end users	Usually defined by developers or other tech experts
Types	Business rules, administrative functions, data management, transaction management, etc.	Usability, reliability, performance, security, scalability, compatibility, etc.

Software Requirements – Non-Functional



Software Requirements Engineering

- A Process of
  - finding
  - analyzing
  - documenting and
  - checking requirements.

Software Requirements Engineering

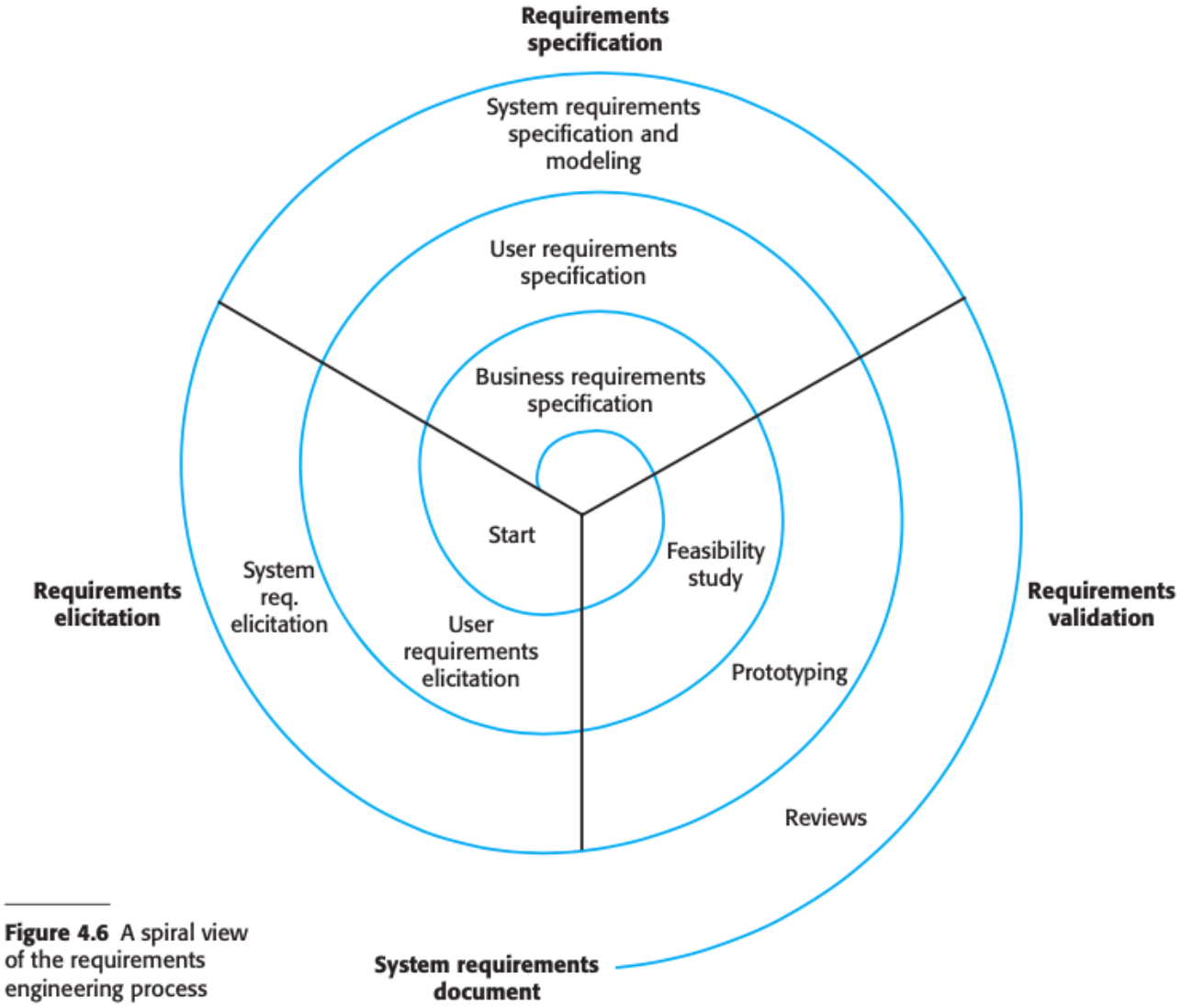


Figure 4.6 A spiral view of the requirements engineering process

Software Requirements Engineering - Activities

- **Feasibility Study:**

- Purpose: To assess the technical, economic, and operational feasibility of the proposed system.

- Activities: Market analysis, technology assessment, risk analysis, cost-benefit analysis.

Software Requirements Engineering - Activities

- **Requirements Elicitation:**

- **Purpose:** To gather information about the needs and expectations of stakeholders (users, customers, domain experts).

- **Techniques**:
  - **Interviews:** One-on-one or group discussions to gather detailed information.
  - **Surveys:** Questionnaires to collect data from a large number of stakeholders.
  - **Observation**: Observing users interacting with existing systems or performing their tasks.
  - **Focus groups**: Group discussions to explore ideas and gather feedback.
  - **Workshops**: Collaborative sessions to brainstorm and refine requirements.
  - **Prototyping**: Creating early versions of the system to gather user feedback.

Software Requirements Engineering - Activities

- **Requirements Analysis:**

- **Purpose:** To analyze the elicited information, identify conflicts, and prioritize requirements.

- **Techniques:**
  - **Use case analysis**: Describing the interactions between users and the system.
  - **Data flow diagrams**: Visualizing the flow of data through the system.
  - **Requirement decomposition**: Breaking down complex requirements into smaller, more manageable ones.
  - **Risk analysis**: Identifying potential risks and developing mitigation strategies.

Software Requirements Engineering - Activities

- **Requirements Specification:**

- **Purpose:** To document the requirements in a clear, concise, and unambiguous manner.

- **Deliverables**: Software Requirements Specification (SRS) document, which includes:
- **Functional requirements**: What the system should do.
- **Non-functional requirements**: Quality attributes of the system (e.g., performance, usability, security).
- **User requirements**: Requirements from the perspective of end-users.
- **System requirements**: Requirements from the perspective of the system developers.

Software Requirements Engineering - Activities

- **Requirements Validation:**

- **Purpose:** To ensure that the requirements are correct, complete, consistent, and feasible.

- **Techniques:**
  - **Reviews and inspections**: Formal or informal reviews by stakeholders and experts.
  - **Prototyping**: Creating prototypes to validate user requirements.
  - **Testing:** Conducting tests to verify that the system meets the specified requirements.

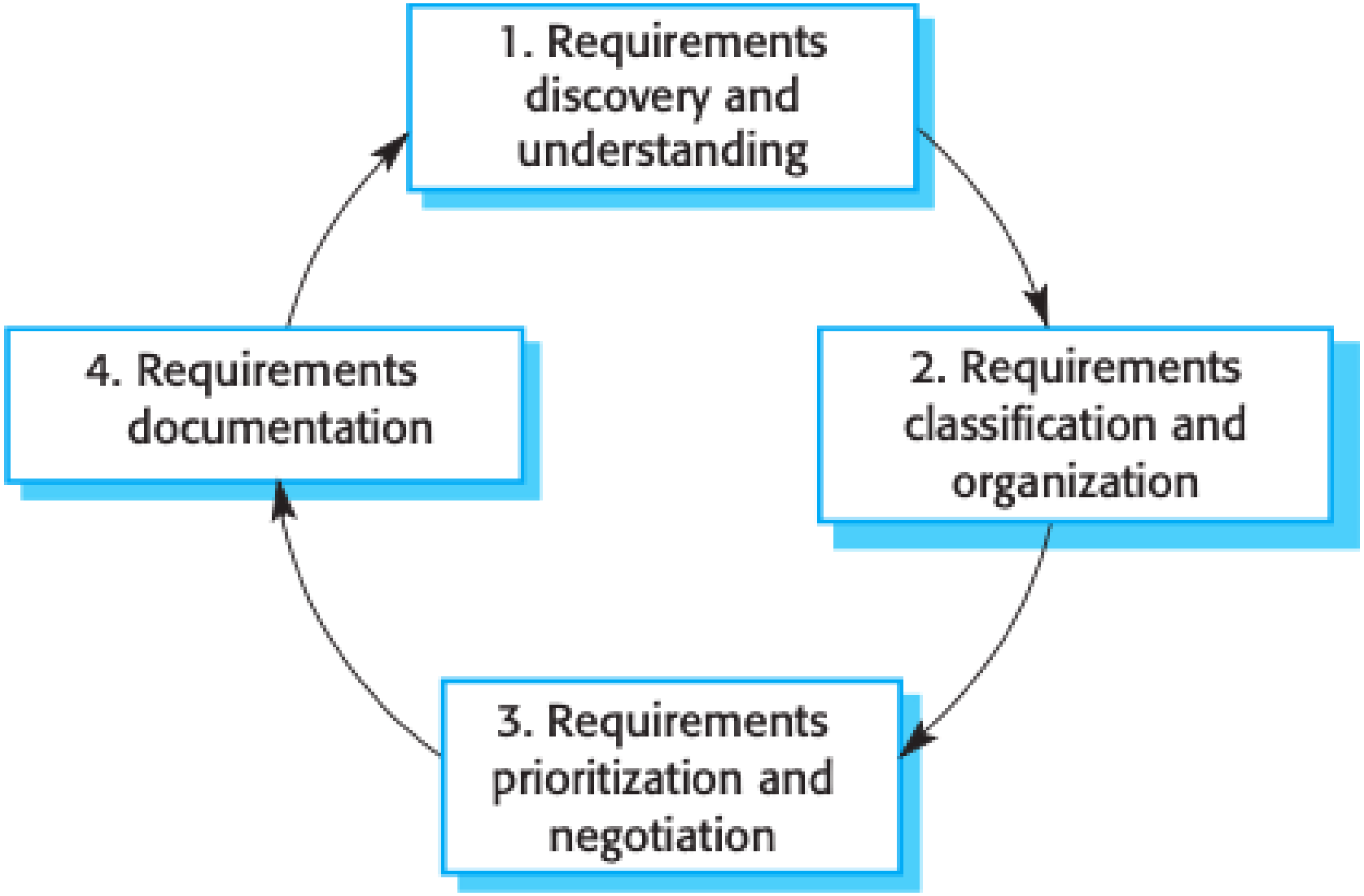
Software Requirements Engineering - Activities

- **Requirements Management:**

- **Purpose:** To track changes to requirements, manage conflicts, and ensure that the system remains aligned with the evolving needs of stakeholders.

- **Activities:**
  - **Change control**: Establishing a process for managing changes to requirements.
  - **Version control**: Maintaining different versions of the requirements documents.
  - **Traceability:** Linking requirements to other project artifacts (e.g., design documents, test cases).

Requirement Elicitation & Analysis Process



Requirement Elicitation & Analysis Process

- **Requirements Discovery and Understanding:**
  - Interact with stakeholders to gather information through interviews, workshops, surveys, etc.
  - Identify domain requirements and constraints.

- **Requirements Classification and Organization:**
  - Group related requirements and organize them into a coherent structure.
  - Use techniques like viewpoints (grouping requirements by stakeholder) or associating requirements with system subsystems.

Requirement Elicitation & Analysis Process

➢Requirements Prioritization and Negotiation:

- ➢ Identify and resolve conflicts among stakeholder requirements.
- ➢ Prioritize requirements based on factors like importance, urgency, and feasibility.
- ➢ Facilitate stakeholder meetings to reach consensus.

➢Requirements Documentation:

- ➢ Document requirements clearly, concisely, and unambiguously.
- ➢ Use simple language and diagrams to enhance understanding.
- ➢ Utilize shared tools like wikis or cloud-based documents for easy access and collaboration.

Requirement Elicitation Techniques & Tools

- **Interview**
  - **Purpose:** To understand stakeholder needs and how they might use a new system.
  - Approaches:
    - **Interviews:**
      - Closed: Predefined set of questions.
      - Open: Exploratory, with no fixed agenda.
    - **Observation/Ethnography:** Observing stakeholders in their work environment.

Requirement Elicitation Techniques & Tools

- **Challenges of Interviewing:**
  - **Stakeholder Uncertainty**: May not clearly articulate needs or may make unrealistic demands.
  - **Communication Gap**: Jargon, implicit knowledge, and differing perspectives can hinder understanding.
  - **Political Factors:** Organizational politics can influence requirements.

- **Effective Interviewing Techniques:**
  - **Open-mindedness**: Be receptive to unexpected requirements.
  - **Active Listening**: Encourage stakeholder input and avoid imposing preconceived notions.
  - **Use of Prompts**: Guide the conversation with questions or scenarios to facilitate discussion.

Requirement Elicitation Techniques & Tools

- **Limitations of Interviews:**
  - May not capture all requirements, especially those that are implicit or difficult to articulate.
  - May not reveal organizational constraints or political influences.

Requirement Elicitation Techniques & Tools

- Ethnography
  - **Observational Technique:** Involves immersing oneself in the work environment of stakeholders.
  - **Focus:** Understanding the actual work practices, social interactions, and organizational context.
  - **Value:**
    - Discovers implicit requirements that may not be explicitly stated.
    - Reveals how people work, not just how they are supposed to work.
    - Provides insights into the social and organizational factors that influence work.

Requirement Elicitation Techniques & Tools

- **Benefits of Ethnography:**

  - **Uncovers Hidden Requirements:**
    - Identifies workarounds and informal practices that may not be documented.
    - Reveals requirements related to cooperation and awareness of other people's activities.

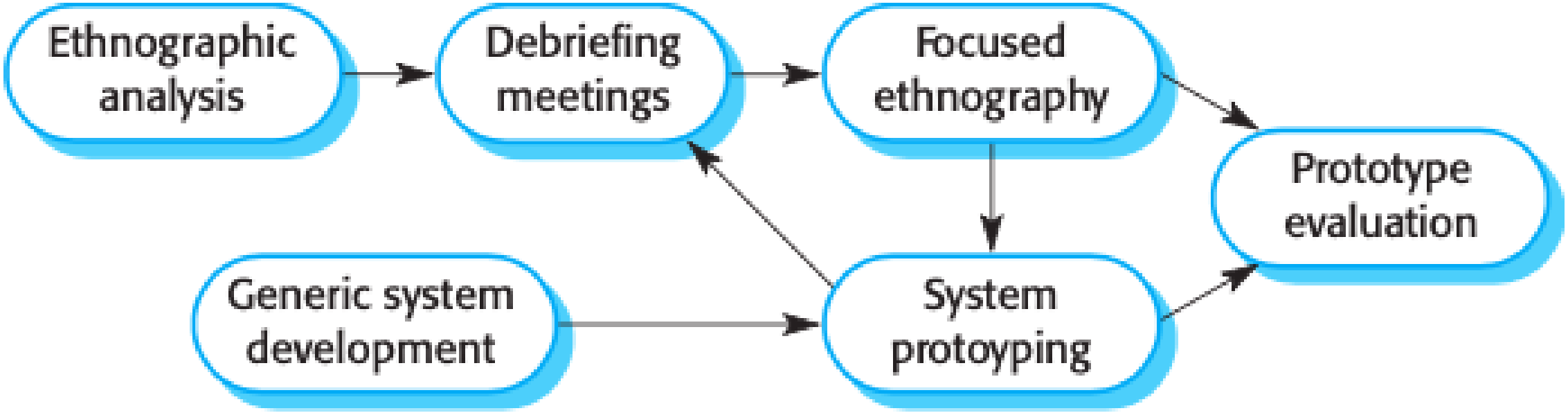
  - **Improves System Design:**
    - Informs prototype development, leading to more user-centered designs.
    - Reduces the number of prototype refinement cycles.

Requirement Elicitation Techniques & Tools

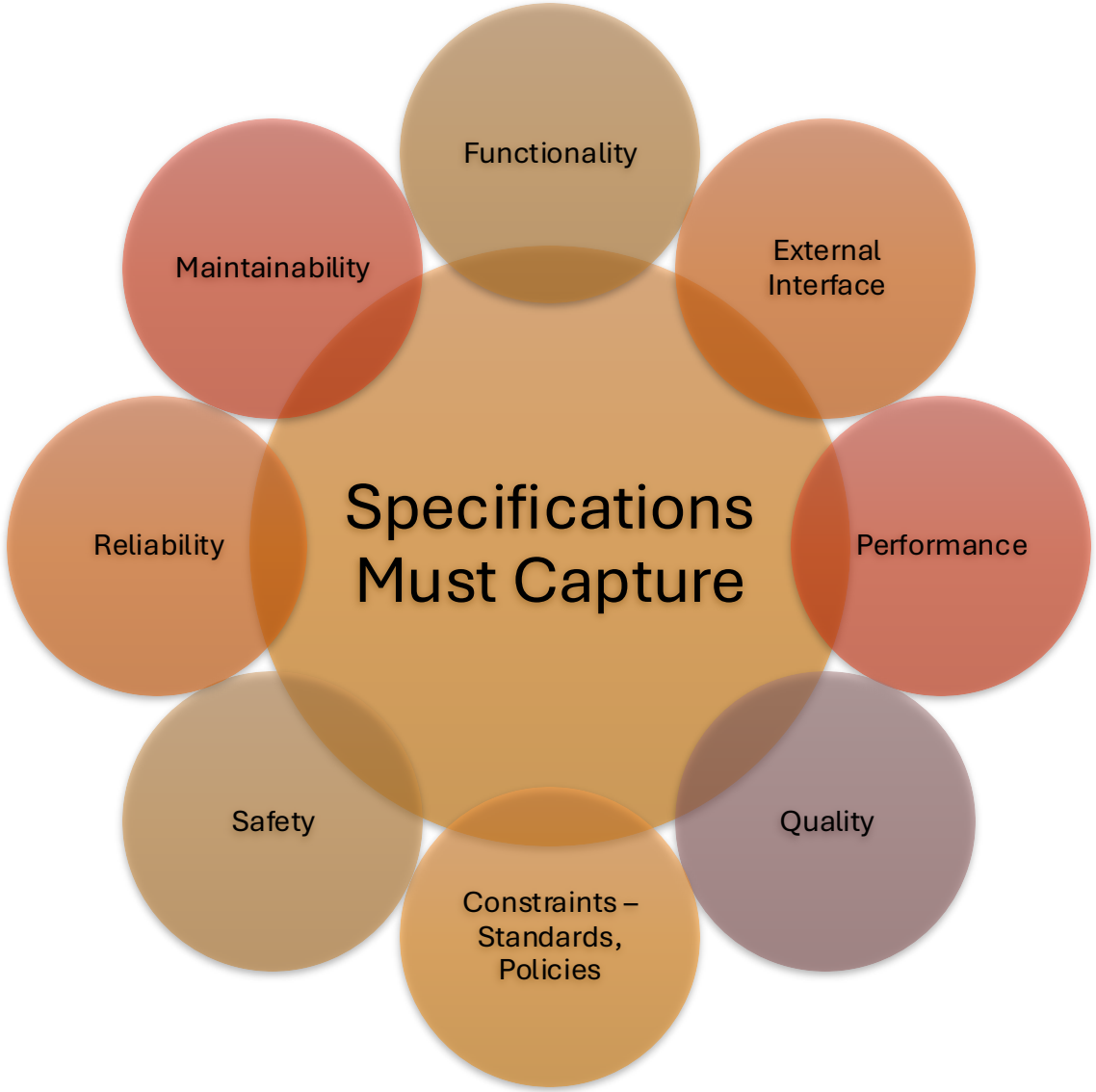
- **Limitations of Ethnography:**

  - **Focus on End-User Perspective:** May not effectively capture broader organizational or domain requirements.
  - **Limited for Innovation:** May not be the best approach for groundbreaking, innovative system designs.
  - **Time-Consuming:** Can be time-intensive and resource-intensive to conduct.

Requirement Elicitation Techniques & Tools



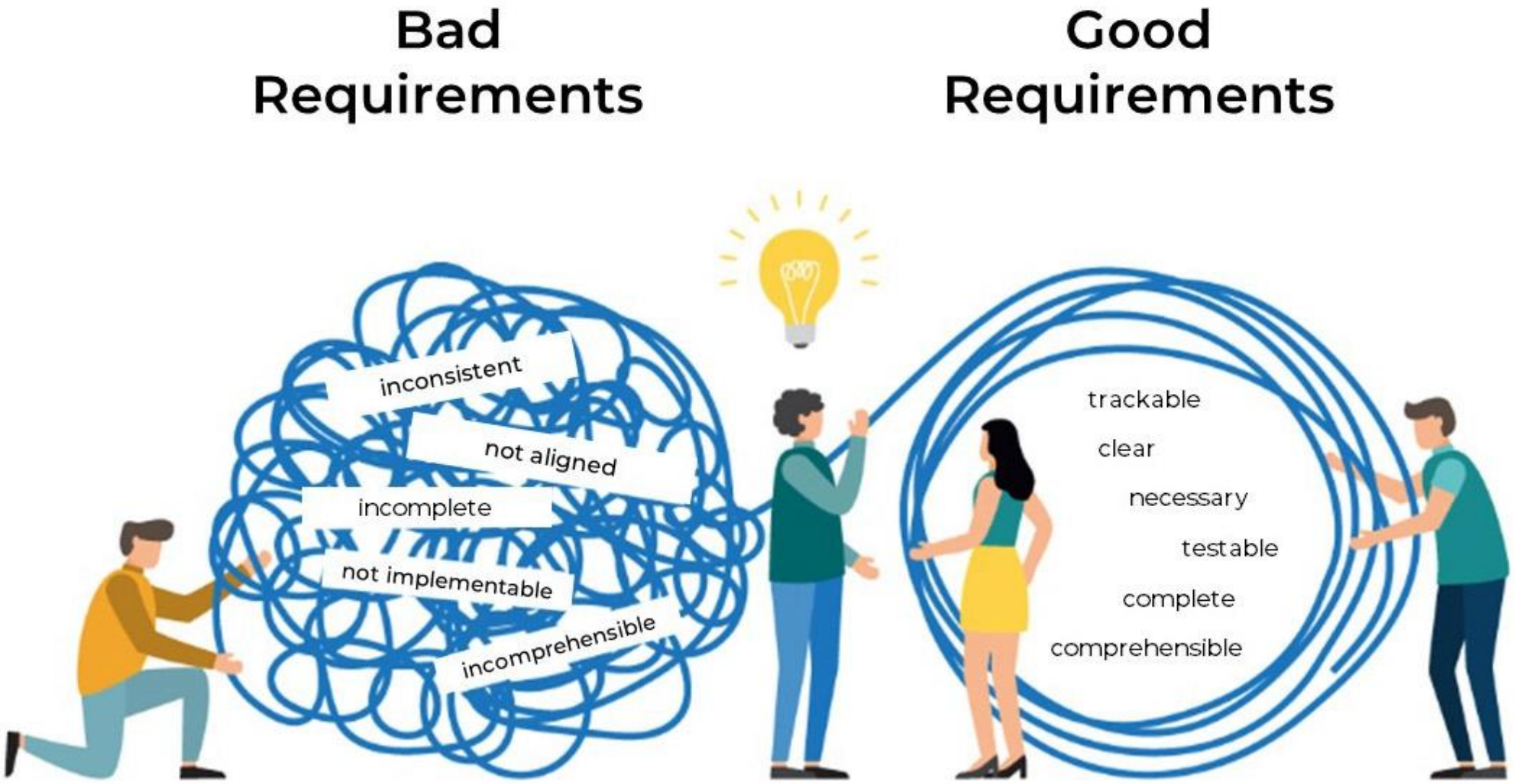
Requirement Specifications



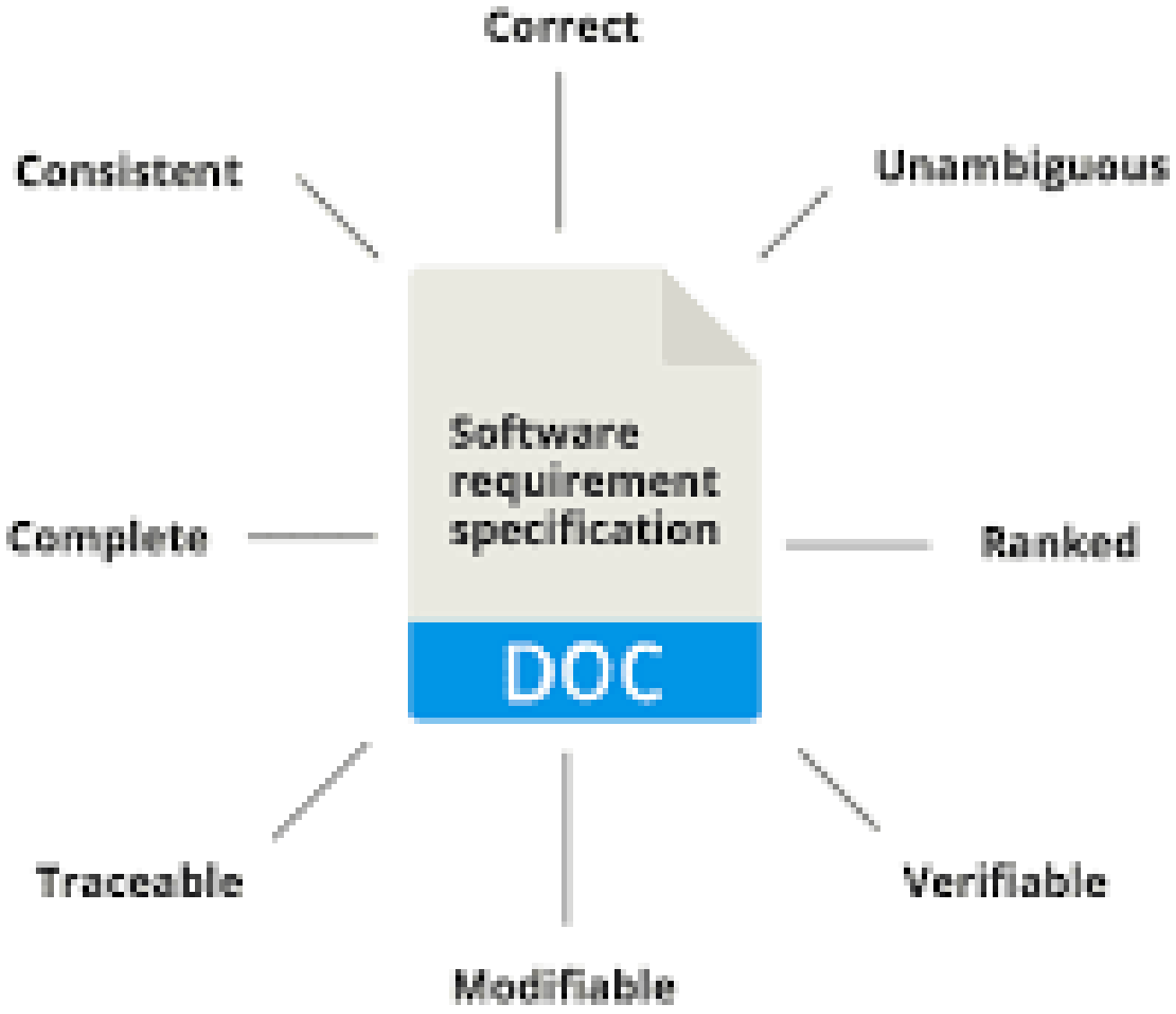
Requirement Specifications – It Captures

- **Functionality**. What is the software supposed to do?
- **External interfaces**. How does the software interact with people, the system's hardware, other hardware, and other software?
- **Performance**. What is the speed, availability, response time, recovery time of various software functions, and so on?
- **Attributes**. What are the portability, correctness, maintainability, security, and other considerations?
- **Design constraints imposed on an implementation**. Are there any required standards in effect, implementation language, policies for database integrity, resource limits, operating environment(s) and so on?

Requirement Specifications – How it should be?



Software Requirement Specifications Characteristics



Software Requirement Specifications Characteristics

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: To Be Determined List 6

Software Requirement Specifications Characteristics

Table of Contents of an SRS

1. Introduction
   1.1 Purpose
   1.2 Scope
   1.3 Definitions, acronyms, and abbreviations
   1.4 References
   1.5 Overview

2. Overall description
   2.1 Product perspective
   2.2 Product functions
   2.3 User characteristics
   2.4 Constraints
   2.5 Assumptions and dependencies

3. Specific requirements *(the largest & most important part of an SRS, organized in different manner)*

Supporting informations
- Table of contents and Index
- Appendixes *(e.g. Samples I/O formats, background, description of problem to be solved by the software)*

Requirement Specifications – How it should be?

- **Valid (or "correct")**
  - Expresses only the real needs of the stakeholders (customers, users,...)
  - Doesn't contain anything that isn't "required"
- **Unambiguous**
  - Every statement can be read in exactly one way
- **Complete**
  - Specifies all the things the system must do and all the things it must not do!
- **Conceptual Completeness**
  - E.g. responses to all classes of input !Structural Completeness
  - E.g. no TBDs!!!
- **Understandable (Clear)**
  - E.g. by non-computer specialists

Requirement Specifications – How it should be?

- **Consistent**
  - Doesn't contradict itself " I.e., is satisfiable
  - Uses all terms consistently
- **Ranked**
  - Must indicate the importance and/or stability of each requirement
- **Verifiable**
  - A process exists to test satisfaction of each requirement
  - every requirement is specified behaviorally"
- **Modifiable**
  - Can be changed without difficulty
  - Good structure and cross-referencing
- **Traceable**
  - Origin of each requirement must be clear
  - Facilitates referencing of requirements in future documentation

Requirement Management in Agile

➢ Stories and Scenarios

- **Purpose:** To capture real-world usage scenarios and elicit system requirements.
- **Stories:**
  - High-level narratives describing how the system might be used.
  - Effective for setting the "big picture" and facilitating broad discussions.
  - Can be used to gather input from a wider audience (e.g., through wikis).
- **Scenarios:**
  - More detailed descriptions of specific user interactions.
  - Typically structured with information about inputs, outputs, normal flow, error handling, and system state.
  - Can be used to derive specific system requirements.

Requirement Management in Agile

- **Relationship between Stories and Scenarios:**
  - Stories provide the overall context, while scenarios delve into specific use cases.
  - Stories can be used to generate more detailed scenarios.

- **Example:**
  - **Story:** A teacher uses the iLearn system to support student projects on the fishing industry.
  - **Scenario:** A student uploads photos to the KidsTakePics system. The teacher moderates the photos before they are shared.

Requirement Management in Agile

- **Benefits of Using Stories and Scenarios:**
  - **Improved Communication:** Facilitates discussions and helps stakeholders understand the system in a more concrete way.
  - **Real-World Focus:** Helps ensure that the system meets the actual needs of users in real-world situations.
  - **Early Identification of Issues:** Allows for early identification and resolution of potential problems.

System Modelling

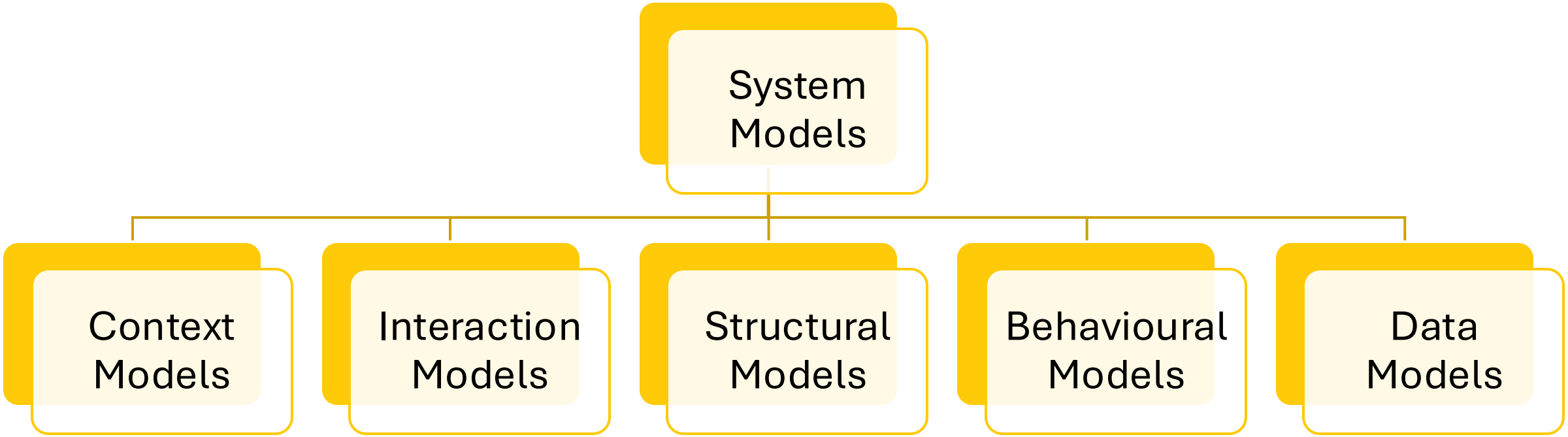
- System modeling refers to the process of developing simplified, abstract representations of a system's components, their interactions, and behaviors.

- These models serve as tools for visualizing, specifying, and validating system requirements and design.

System Modelling - Purpose

- **Understanding and Analysis**: System models help stakeholders comprehend the **structure and behavior of the system**, breaking down complexity into manageable parts.
- **Communication**: They provide a **common language** that facilitates **discussions among developers, clients, and other stakeholders**, ensuring everyone has a **shared understanding of the system**.
- **Specification**: Models **capture essential requirements and functionalities**, guiding the development process.
- **Validation**: They **enable testing of the system's design against requirements** to ensure it meets stakeholder expectations.

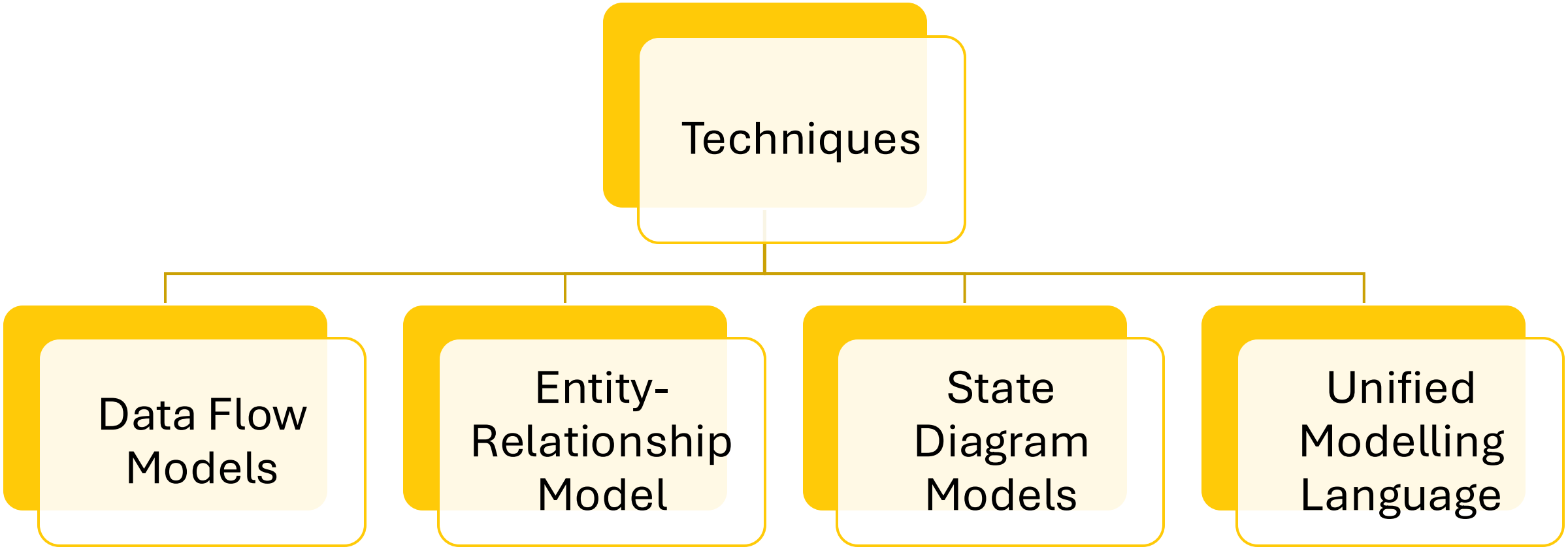
System Modelling - Models



System Modelling - Models

- **Context Models:** Illustrate the **system's environment and its interactions with external entities** to define system boundaries.
- **Interaction Models**: Focus on how **components within the system communicate and interact** with each other.
- **Structural Models**: Represent the **organization of the system's components and their relationships**, often using class diagrams or entity-relationship diagrams.
- **Behavioral Models**: Capture how the **system behaves in response to various inputs or events**, using state machine diagrams or sequence diagrams.
- Data Models: Represents the structure and organization of data within the system

System Modelling – Modelling Techniques



System Modelling - Techniques

- Context Models – UML & DFD
- Interaction Model – UML
- Structural Model – UML & ERD
- Behavioral Model – UML
- Data Model – ERD & UML

System Modelling – Data Flow Diagram Model

- A Data Flow Diagram (DFD) is a graphical representation that **illustrates the flow of data within a system**.

- It serves as a powerful tool for **visualizing and understanding how information is processed, stored, and communicated**, making it essential in system analysis and design.

System Modelling – Data Flow Diagram Model

Purpose and Benefits

- **Visualization:** DFDs provide a clear and concise visual representation of how data flows through a system, making it easier to understand complex processes.

- **Identification of Bottlenecks**: By mapping out the flow of information, DFDs help identify potential inefficiencies or bottlenecks in data processing.

- **Documentation:** DFDs serve as documentation tools that communicate system designs to various stakeholders, from technical teams to non-technical audiences.

- **System Analysis**: They can be used to analyze existing systems or model new ones, facilitating discussions around improvements or redesigns.

System Modelling – Data Flow Diagram Model

DFDs consist of four primary components:

- **Processes:** Represented by rounded rectangles, processes transform input data into output data. Each process is typically labeled with a verb phrase that describes its function.
- **Data Flows:** Illustrated as arrows, data flows indicate the movement of data between processes, data stores, and external entities. They are labeled with descriptions of the data being transferred.
- **Data Stores (Warehouses)**: Depicted as open-ended rectangles, these represent locations where data is stored for later use. Data can flow into and out of these stores as needed.
- **External Entities (Terminators)**: Shown as squares or rectangles, external entities are sources or destinations of data outside the system being modeled. They interact with the system by providing inputs or receiving outputs.

System Modelling – Data Flow Diagram Model

When creating a DFD, several rules should be followed:

- Identify the key processes and external entities involved.
- Determine the input and output data for each process.
- Use standardized symbols to represent components clearly.
- Label all arrows with the type of data being transferred.
- Keep the diagram simple to enhance understanding and avoid complexity.

System Modelling – Data Flow Diagram Model

Advantages:

- Provides a clear overview of data flow within a system.
- Helps in identifying potential issues in data handling.
- Facilitates communication among stakeholders regarding system design.

Disadvantages:

- Can become complex if too many elements are included.
- May require significant time to create and maintain.
- Might not capture dynamic aspects of system behavior effectively.

System Modelling – Data Flow Diagram Model

- **Levels of DFDs**
- **Context Diagram (Level 0)**: The highest-level DFD, showing the system as a single process with external entities.
- **Level 1 DFD**: Decomposes the context diagram into major processes and data flows.
- **Level 2 DFD**: Further decomposes Level 1 processes into more detailed processes and data flows.
- **Level 3 DFD**: Decomposes Level 2 processes into even more detailed processes and data flows.

System Modelling – Data Flow Diagram Model

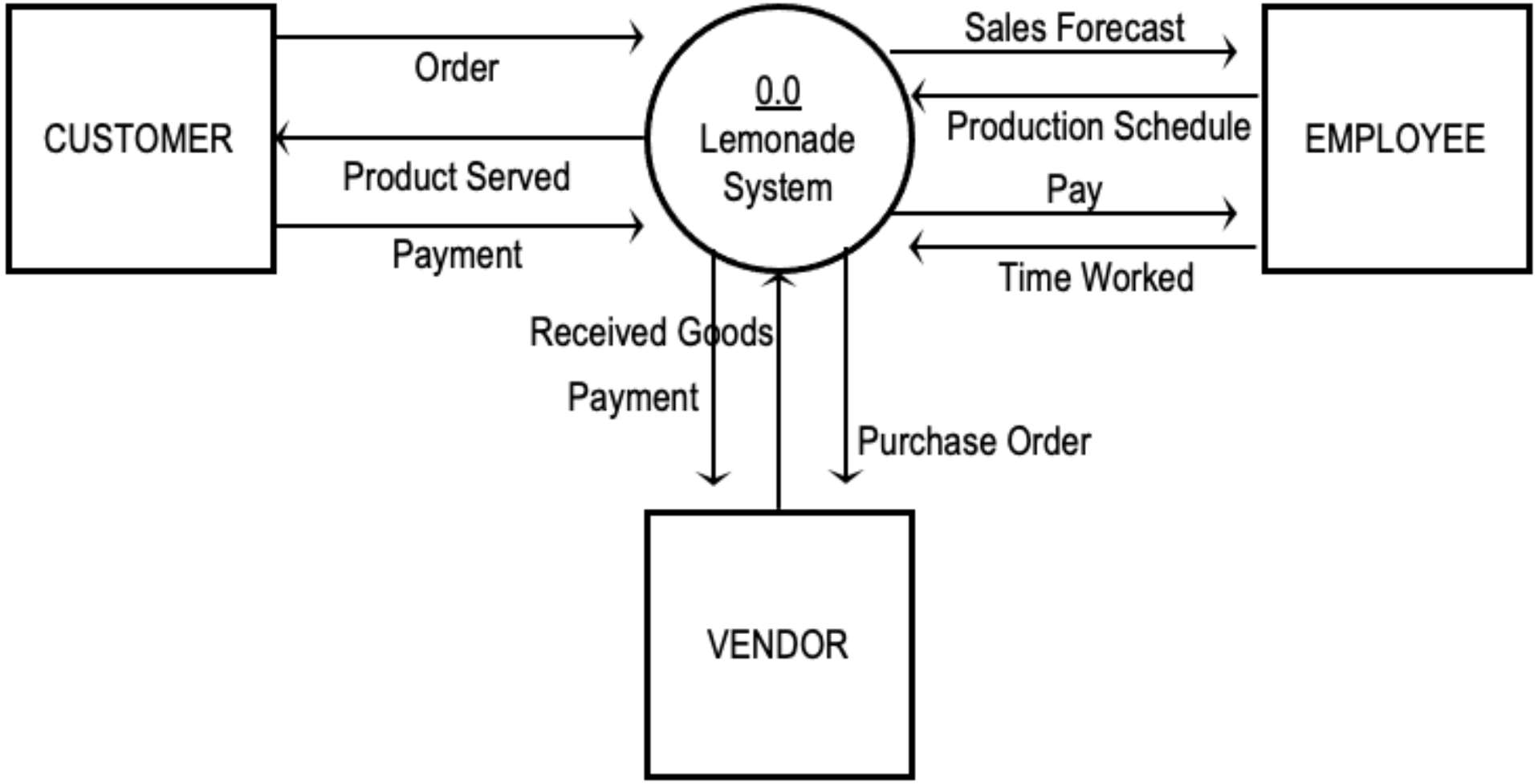
- Visualization of Data Flow: DFDs provide a graphical representation of how data moves through a system, illustrating processes, data stores, and external entities. This visualization helps stakeholders understand how information is processed and transferred, which is essential for accurately capturing requirements16.
- Clarification of System Boundaries: By defining what is inside and outside the system, DFDs help delineate the scope of the project. This clarity is vital during requirement elicitation to ensure that all necessary functionalities are identified and documented34.
- Facilitation of Communication: DFDs serve as a communication tool among stakeholders, including developers, clients, and users. They provide a common framework for discussing system processes, ensuring that everyone has a shared understanding of how data flows within the system

System Modelling – Data Flow Diagram Model

- Example - Lemonade System
- The Lemonade System involves the following processes:
  - **Customer Orders**: A customer places an order for lemonade.
  - **Make Lemonade**: The lemonade stand owner makes the lemonade according to the customer's request.
  - **Payment**: The customer pays for the lemonade.
  - **Serve Lemonade**: The lemonade stand owner serves the lemonade to the customer.

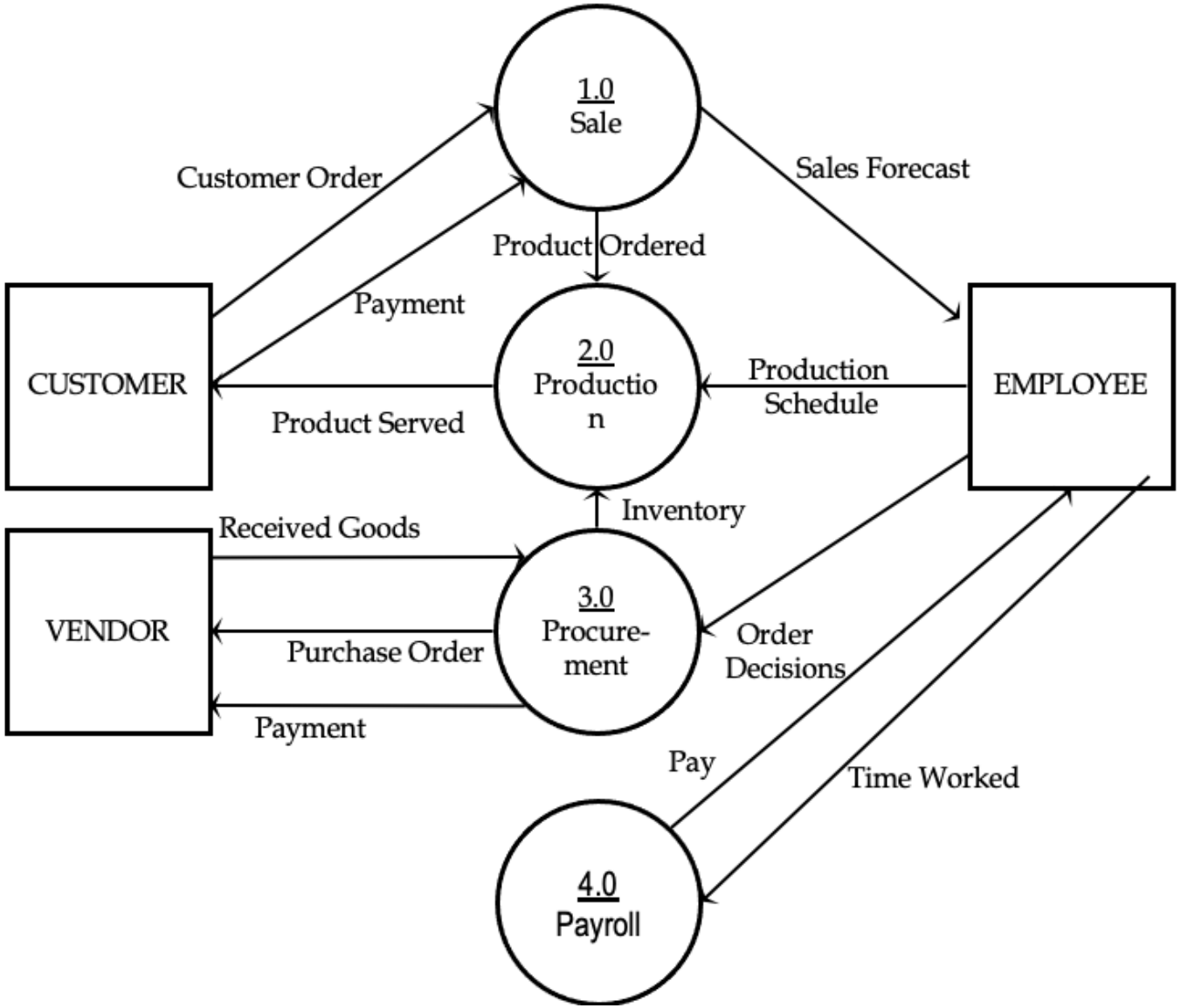
System Modelling – Data Flow Diagram Model

- Context Model – Level 0



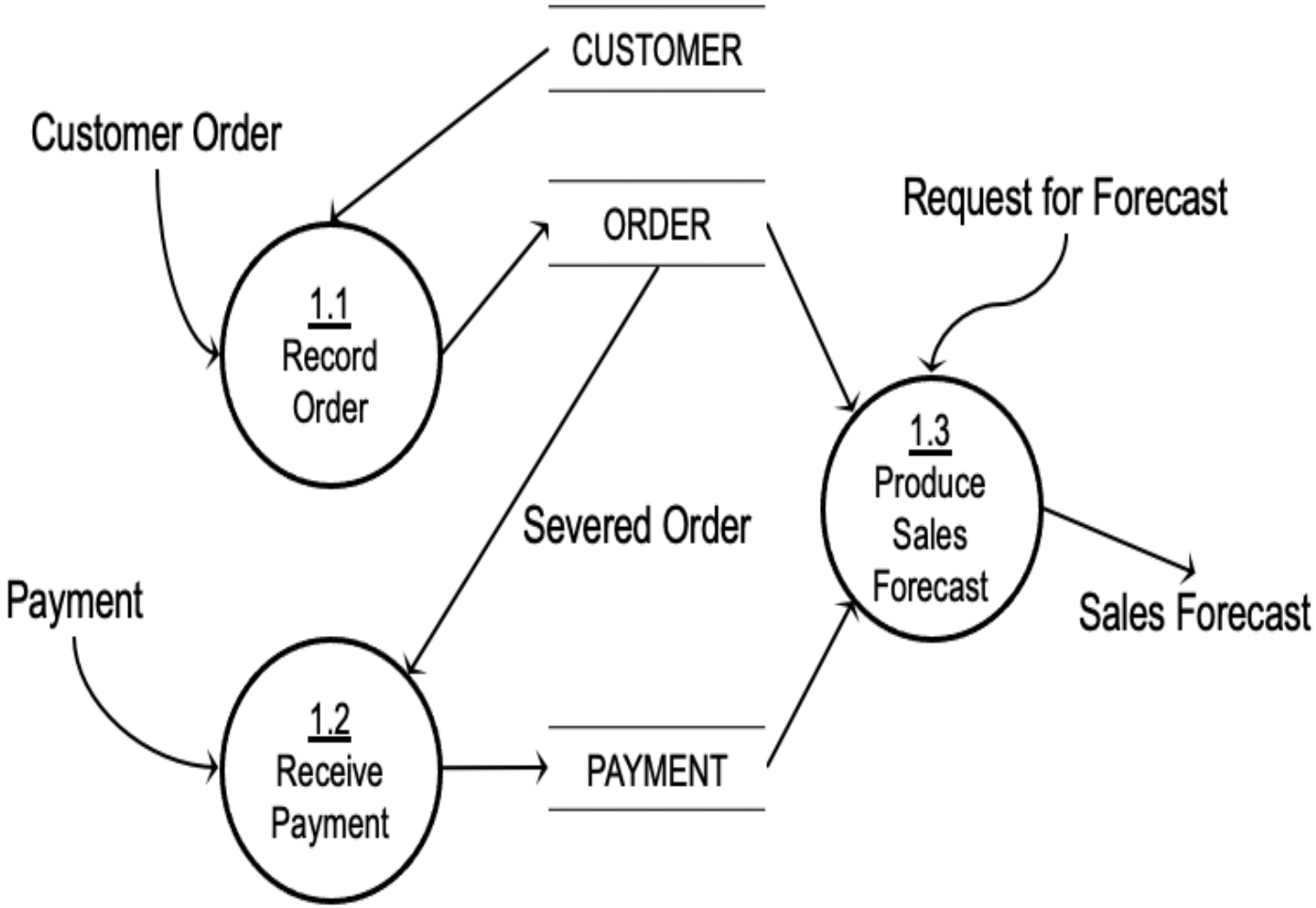
System Modelling – Data Flow Diagram Model

- Level 0 DFD – Identifying manageable sub processes



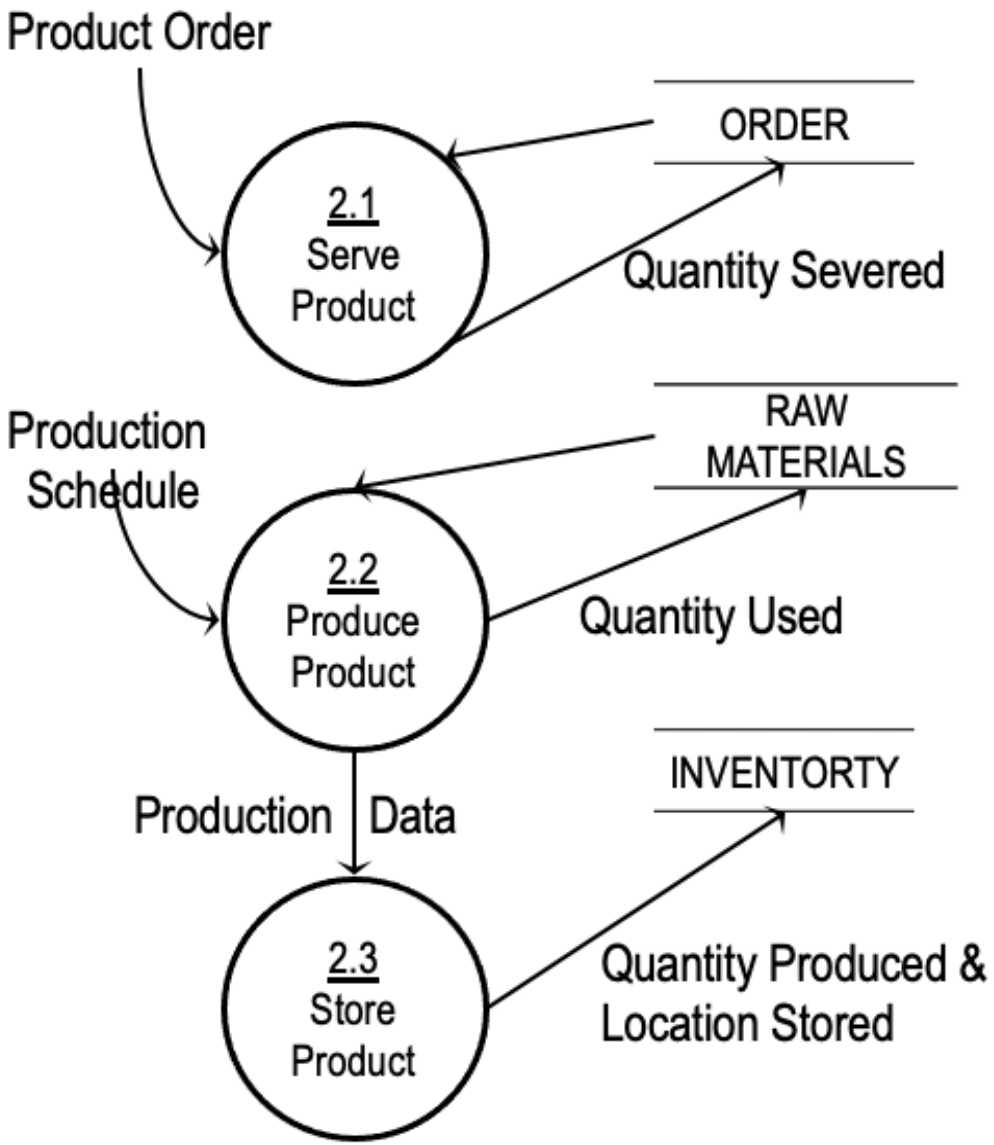
System Modelling – Data Flow Diagram Model

- Level 1 DFD
- Customer Order
- Serve Product
- Collect Payment
- Produce Product
- Store Product
- Order Raw Materials
- Pay for Raw Materials
- Pay for Labor



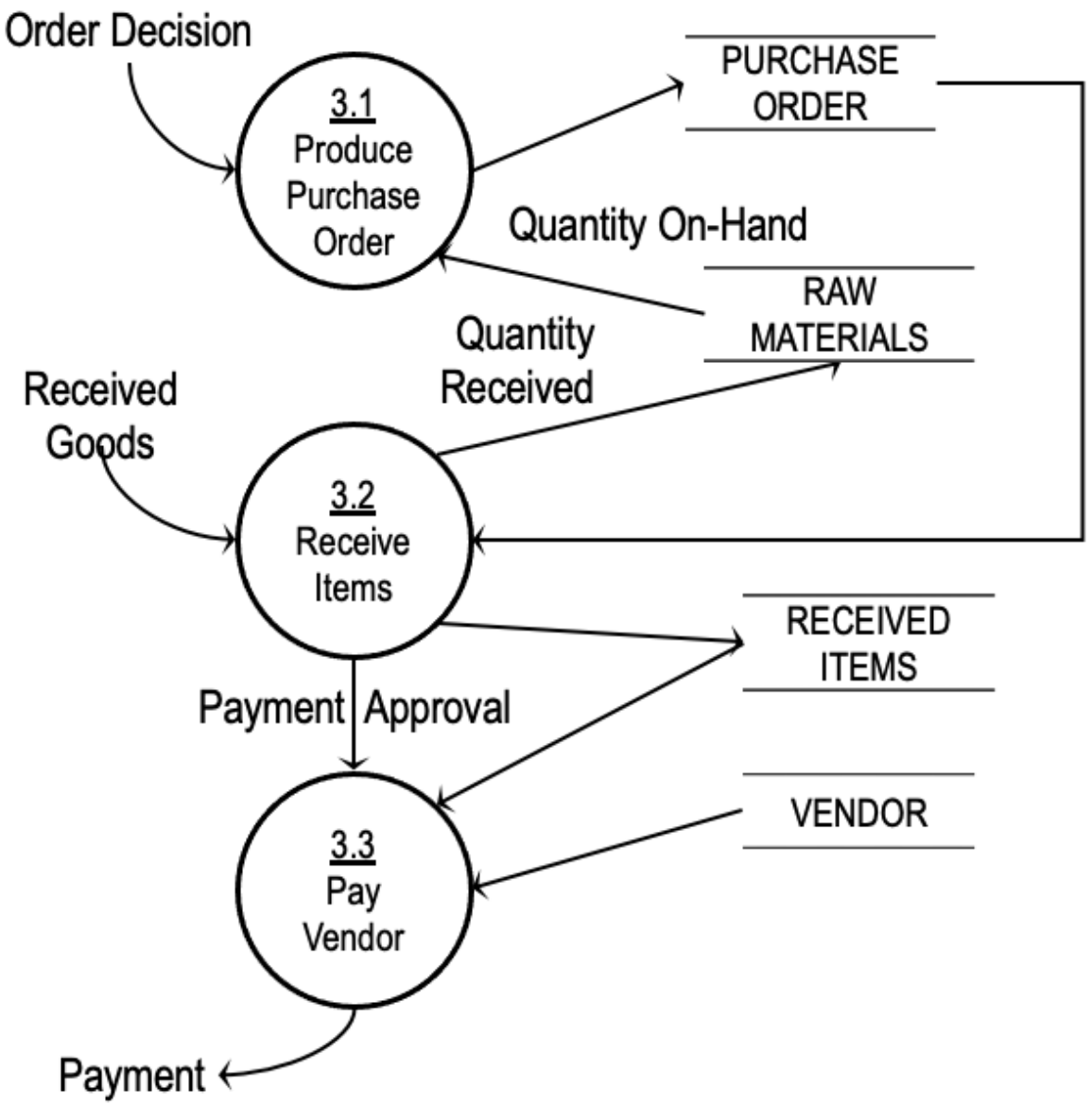
System Modelling – Data Flow Diagram Model

- Level 1 DFD
- Customer Order
- Serve Product
- Collect Payment
- Produce Product
- Store Product
- Order Raw Materials
- Pay for Raw Materials
- Pay for Labor



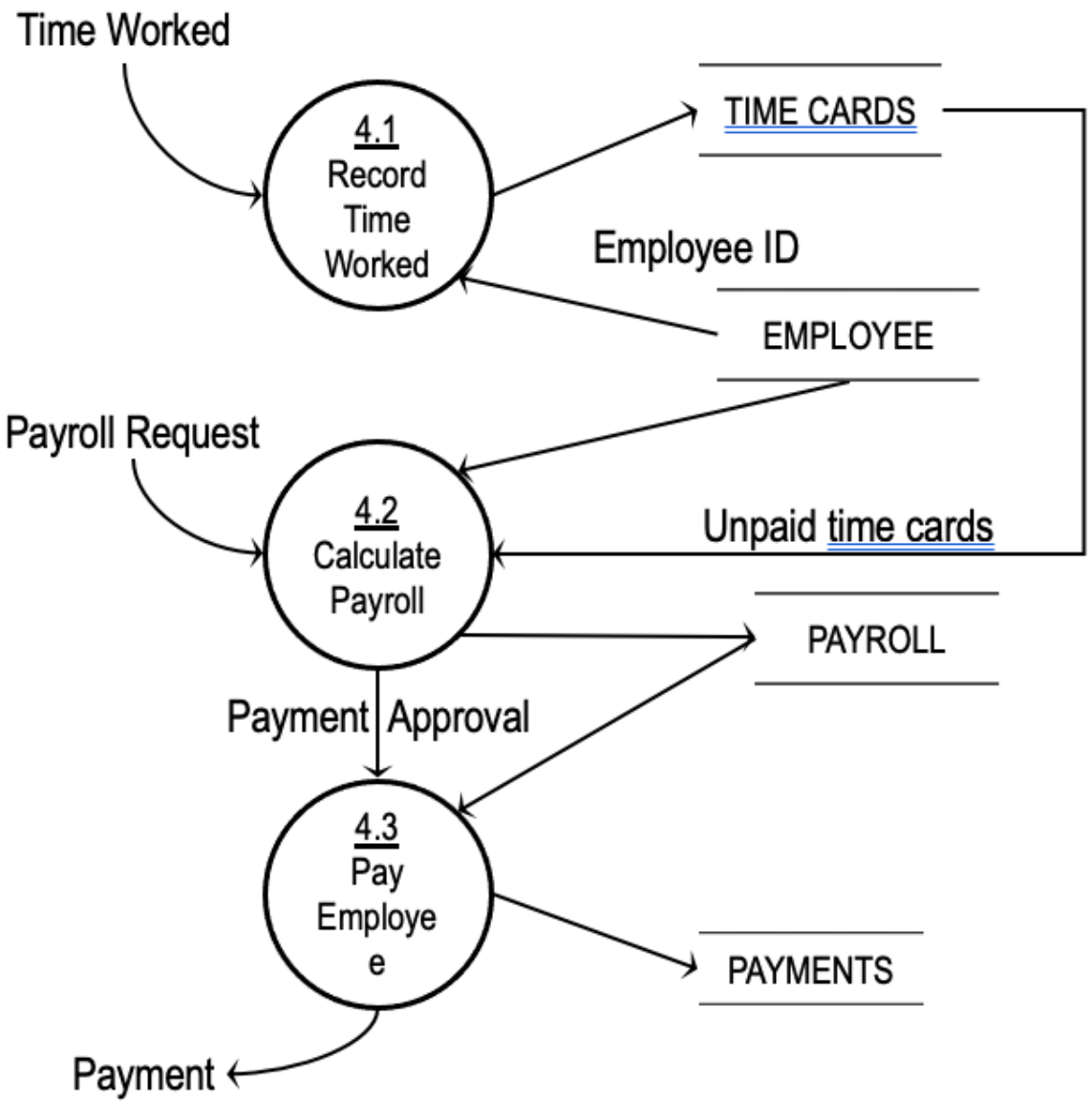
System Modelling – Data Flow Diagram Model

- Level 1 DFD
- Customer Order
- Serve Product
- Collect Payment
- Produce Product
- Store Product
- Order Raw Materials
- Pay for Raw Materials
- Pay for Labor



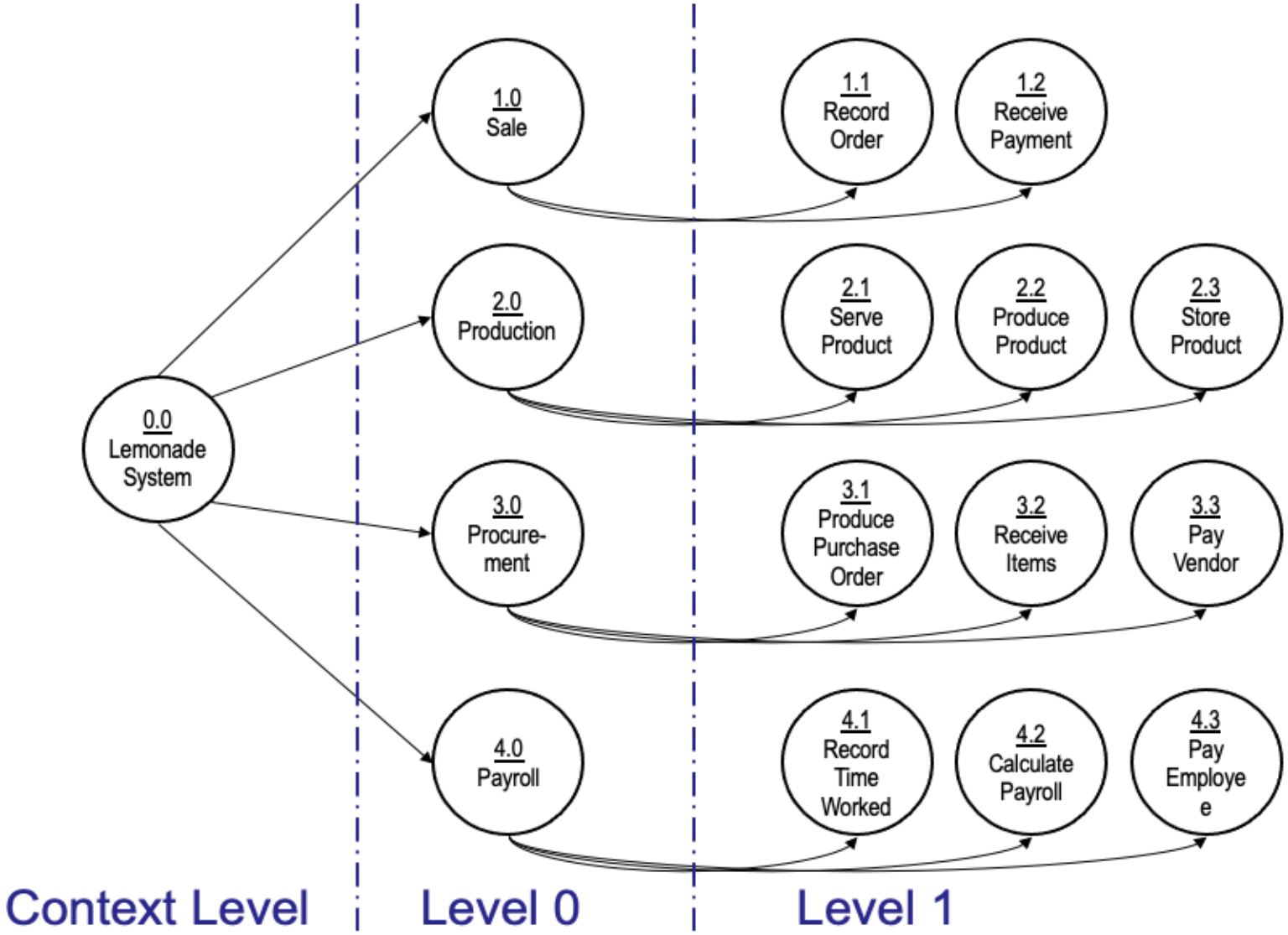
System Modelling – Data Flow Diagram Model

- Level 1 DFD –
- Customer Order
- Serve Product
- Collect Payment
- Produce Product
- Store Product
- Order Raw Materials
- Pay for Raw Materials
- Pay for Labor



System Modelling – Data Flow Diagram Model

- Process Decomposition



System Modelling – Entity Relationship Model

- Represents the logical design of a database by abstracting real-world objects as entities and their associations as relationships.

- Example: Entities "Student" and "School" with the relationship "Studies in."

System Modelling – Entity Relationship Model

- Entity Set & Relationship Set:
- **Entity Set**: A collection of similar entities (e.g., "Student" entity set). Individual instances are entities (e.g., "Ram").
- **Relationship Set**: A collection of similar relationships between entities.
- **Attributes**: Characteristics describing an entity (e.g., Student attributes: name, age, address, roll number, grade).

System Modelling – Entity Relationship Model

- Keys:

- **Super Key**: One or more attributes that uniquely identify an entity (e.g., grade + roll number in a school).
- **Candidate Key**: A minimal subset of a super key (super key without extraneous attributes).
- **Primary Key**: A chosen candidate key for database implementation.
- **Prime Attribute**: Any attribute part of a super key.

System Modelling – Entity Relationship Model

- **Weak Entity**: An entity dependent on another entity for its identification (doesn't have its own super key).
- Example: "Family" (weak entity) dependent on "Employee" (owning entity) for identification.

- **Entity Generalization/Specialization**: Represents hierarchical relationships between entity sets. A generalized entity (e.g., "Person") can have specialized subtypes (e.g., "Customer," "Employee"). Uses "ISA" hierarchy.

System Modelling – Entity Relationship Model

- Mapping Cardinalities (Relationships between Entity Sets E1 and E2):
- **One-to-One**: One entity in E1 relates to at most one entity in E2, and vice versa.
- **One-to-Many**: One entity in E1 relates to zero or more entities in E2. One entity in E2 relates to at most one entity in E1.
- **Many-to-One**: Many entities in E1 relate to one entity in E2. One entity in E2 relates to at most one entity in E1.
- **Many-to-Many**: Many entities in E1 relate to many entities in E2.

System Modelling – Entity Relationship Model

- ER Diagram
  - From a given problem statement, we identify the possible entity sets, their attributes, and relationships among different entity sets.
  - Once we have these information, we represent them pictorially, called an entity-relationship (ER) diagram.

System Modelling – Entity Relationship Notations

Term	Notation	Remarks
Entity set		Name of the set is written inside the rectangle
Attribute		Name of the attribute is written inside the ellipse
Entity with attributes	Name, Roll, Dept, Student	Roll is the primary key; denoted with an underline
Weak entity set		
Relationship set		Name of the relationship is written inside the diamond
Related enity sets	Student, Studies in, School	
Relationship cardinality	Person, 1, Owns, N, Car	A person can own zero or more cars but no two persons can own the same car
Relationship with weak entity set	Employee, Can insure, Spouse	

System Modelling – Entity Relationship Importance

- **Foundation for Database Design**: ER modeling is the crucial first step in designing a relational database. It provides a clear blueprint for translating real-world concepts into database structures.

- **Identifies Key Elements**: ER modeling helps pinpoint the essential components of a database: entities (objects), attributes (properties), and relationships (associations).

- **Visual Representation**: ER diagrams offer a visual representation of the database structure, making it easier to understand and communicate the design.

System Modelling – Entity Relationship Importance

- **Facilitates Table Creation**: The ER diagram directly informs the creation of database tables, including columns (attributes) and connections (relationships).
- **Supports Constraint Definition**: ER modeling aids in defining constraints (rules) on the data, ensuring data integrity and consistency.
- **Enables Database Normalization**: A well-defined ER model is essential for effective database normalization, which reduces data redundancy and improves data management.
- **Detailed and Accurate Design**: The importance of ER modeling lies in its ability to capture a detailed and accurate representation of the data requirements, which is crucial for efficient data storage and retrieval. A good ER diagram leads to a good database.

System Modelling – Entity Relationship Example

- Problem 1

A manufacturing company produces products. The following product information is stored: product name, product ID and quantity on hand. These products are made up of many components. Each component can be supplied by one or more suppliers. The following component information is kept: component ID, name, description, suppliers who supply them, and products in which they are used.

Create an ERD to show how you would track this information.

System Modelling – Entity Relationship Example

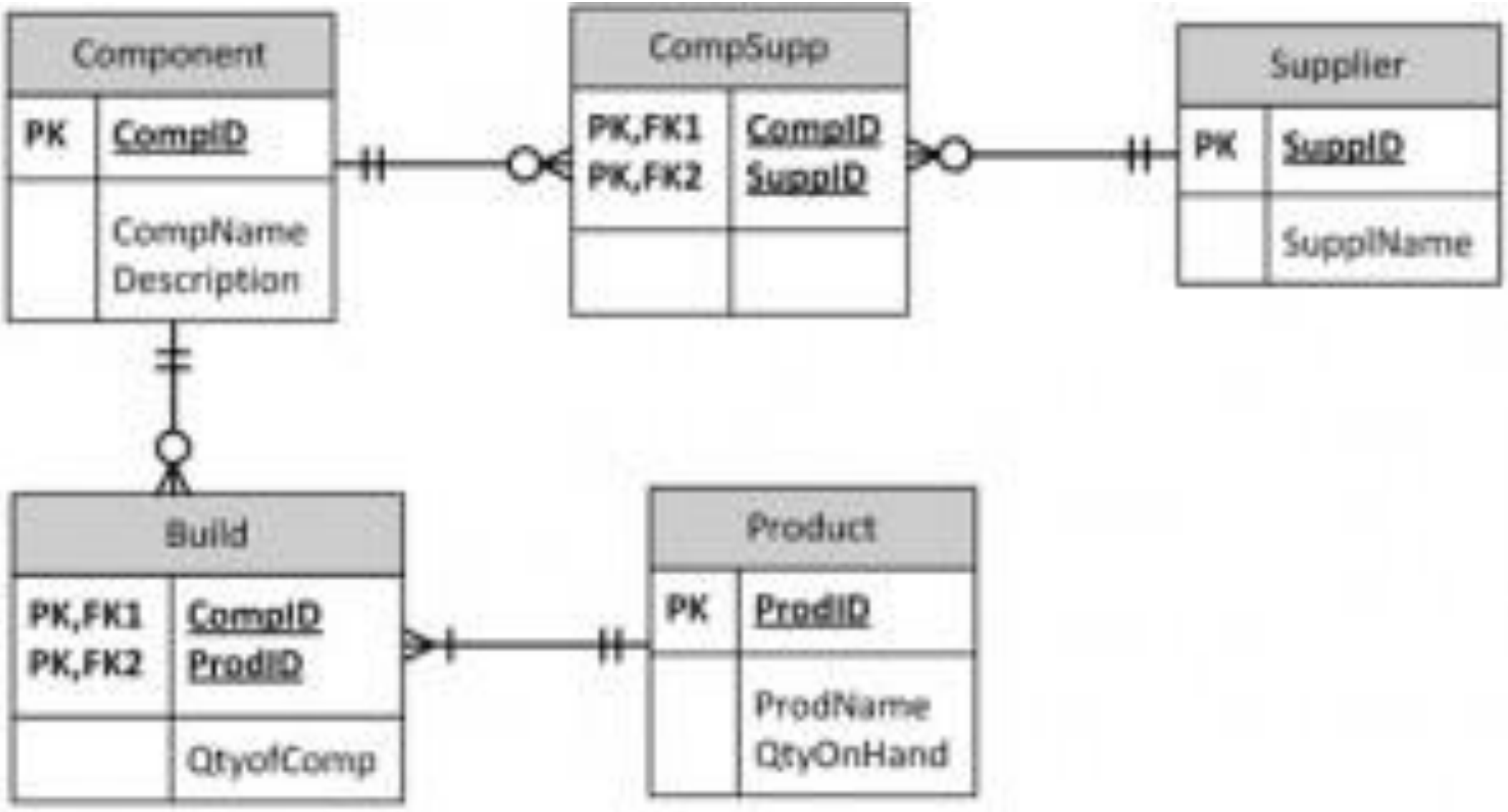
Assumptions

- A supplier can exist without providing components.
- A component does not have to be associated with a supplier.
- A component does not have to be associated with a product. Not all components are used in products.
- A product cannot exist without components.

System Modelling – Entity Relationship Example

- Component(CompID, CompName, Description) PK=CompID
- Product(ProdID, ProdName, QtyOnHand) PK=ProdID
- Supplier(SuppID, SuppName) PK = SuppID
- CompSupp(CompID, SuppID) PK = CompID, SuppID
- Build(CompID, ProdID, QtyOfComp) PK= CompID, ProdID

System Modelling – Entity Relationship Example



System Modelling – Unified Modelling Language (UML)

- A standardized modeling language used in software engineering for **visualizing, specifying, constructing**, and **documenting** the artifacts of software systems.

System Modelling – Unified Modelling Language (UML)

- **Visualization:**
  - UML provides a way to represent software systems graphically, making complex designs easier to understand.
  - Instead of just lines of code, developers can use diagrams to see the structure and behavior of their software.

System Modelling – Unified Modelling Language (UML)

- **Specification:**
  - UML can be used to specify the components of a software system in a precise and unambiguous way.
  - This helps ensure that everyone on the development team has a shared understanding of the system's design

System Modelling – Unified Modelling Language (UML)

- **Construction**: While UML itself isn't code, it can be used to generate code in various programming languages. This can speed up the development process and reduce errors.

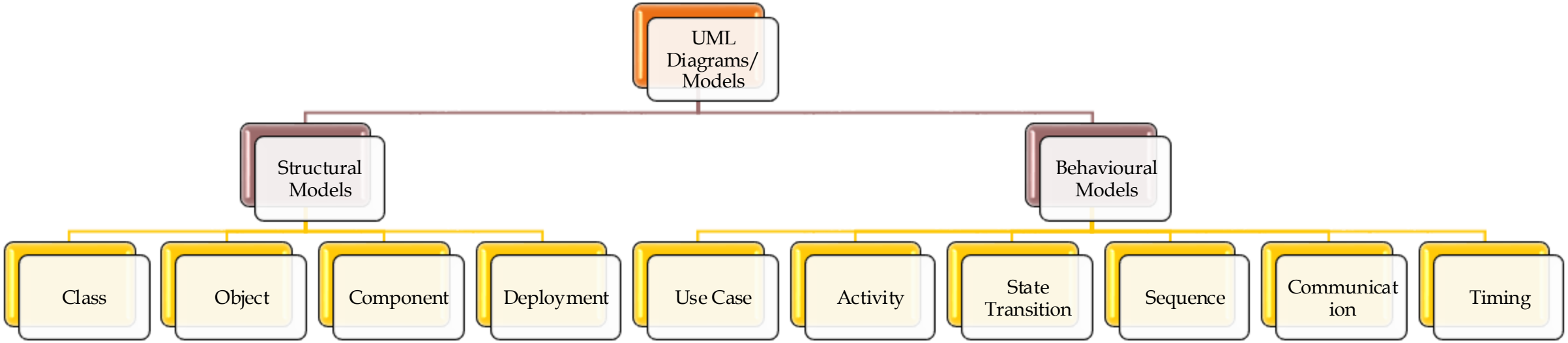
- **Documentation**: UML diagrams serve as excellent documentation for software systems. They provide a clear and concise overview of the system's design, which is helpful for maintenance and future development.

System Modelling – UML - Perspectives

- Structural Model (Static View) - Class Diagram, Component Diagram, Deployment Diagram, Object Diagram

- Behavioral Model (Dynamic View) – Sequence Diagram, Collaboration Diagram, State Chart Diagram, Activity Diagram
  - User Interaction View – Usecase Diagram

System Modelling – UML - Perspectives



System Modelling – UML – Usecase Model

- Provide a shared understanding of system functionality among end-users, developers, and domain experts.
- Capture system functionality (use cases) and users (actors) from a problem statement.
- Behavioral UML diagram showing a graphical overview of system functionality.
- Consist of use cases (actions), actors (users/external systems), and their relationships.

System Modelling – UML – Usecase Model

- **Actor**

- External entity (human, device, or system) interacting with the system for a purpose.

  - **Primary Actor**: Principal user achieving goals through system services (e.g., customer withdrawing cash).

  - **Supporting Actor**: Provides services to the system (e.g., bank representative restocking ATM). Primary actors typically on the top left of the diagram.

System Modelling – UML – Usecase Model

- Use Case:

- A specific functionality provided by the system (e.g., withdraw cash, check balance).

- Includes both successful and unsuccessful interaction scenarios.

System Modelling – UML – Usecase Model

- **Subject:**
- The system under consideration (e.g., ATM).
- Use cases apply to the subject.

System Modelling – UML – Usecase Model

- **Association (Actor-Use Case):**

- Connection between actors and use cases, showing interaction (message passing).
- Actors must be associated with at least one use case, and vice versa.
- Actor associations are not usually shown, but class hierarchies can be depicted.

System Modelling – UML – Usecase Model

- **Use Case Relationships:**
- **Include**: Represents shared behavior among use cases (like a function call). Dashed arrow with «include» stereotype from including to included use case. Example: "Login" included by "Send Email," "Reply," and "Forward."

- **Extend**: Represents variations or optional behaviors of a use case. Dashed arrow with «extend» stereotype from extending to extended use case. Example: "Add Shipping Instructions" extends "Place Order."

- **Generalization**: Represents inheritance between use cases (specialization). Solid arrow from specialized to generalized use case. Example: "Draw Rectangle" generalizes "Draw Polygon."

System Modelling – UML – Usecase Model

- **Identifying Actors:**
  - Who benefits most from the system? (Primary actors)
  - Who keeps the system working?
  - What other software/hardware interacts with the system?
  - Any interface with other systems?
- **Identifying Use Cases:**
  - Determine actor goals (functionality they want).
  - Use case names start with a verb (e.g., "Check Balance").

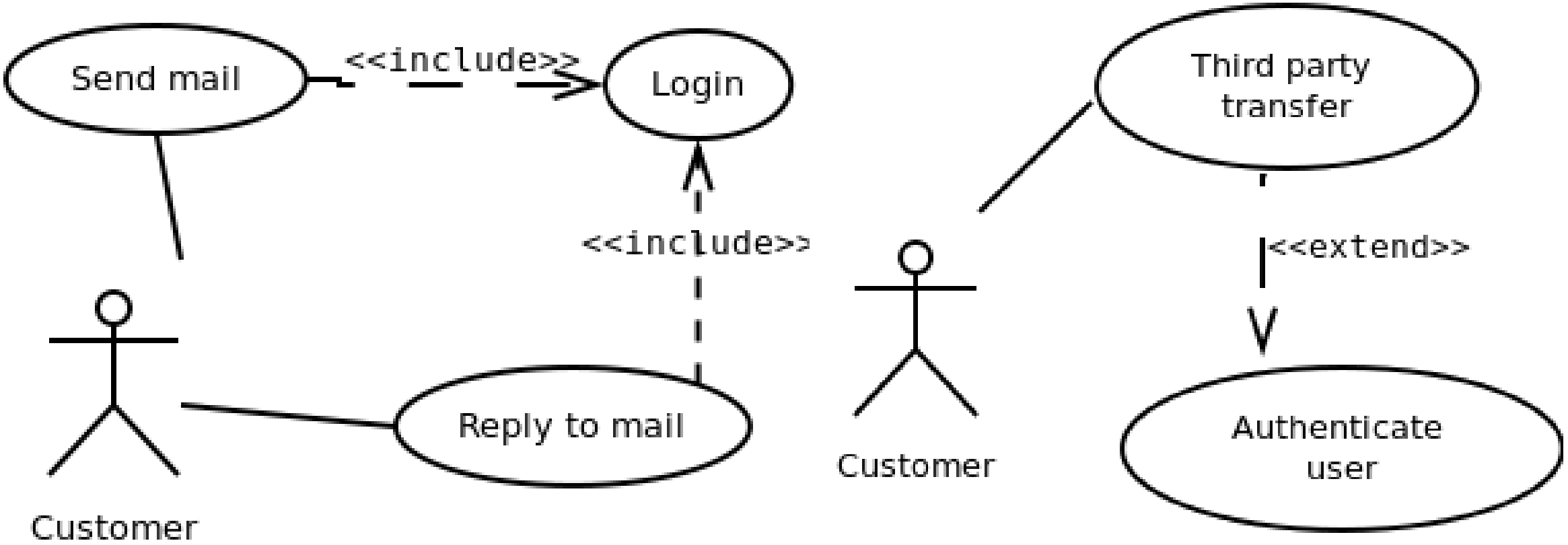
System Modelling – UML – Usecase Model

➢Guidelines for Drawing Use Case Diagrams:

- ➢ Define the system boundary.
- ➢ Actors should have well-defined purposes.
- ➢ Use cases should represent meaningful work for actors.
- ➢ Connect all actors and use cases.
- ➢ Use "include" relationships for shared behavior.

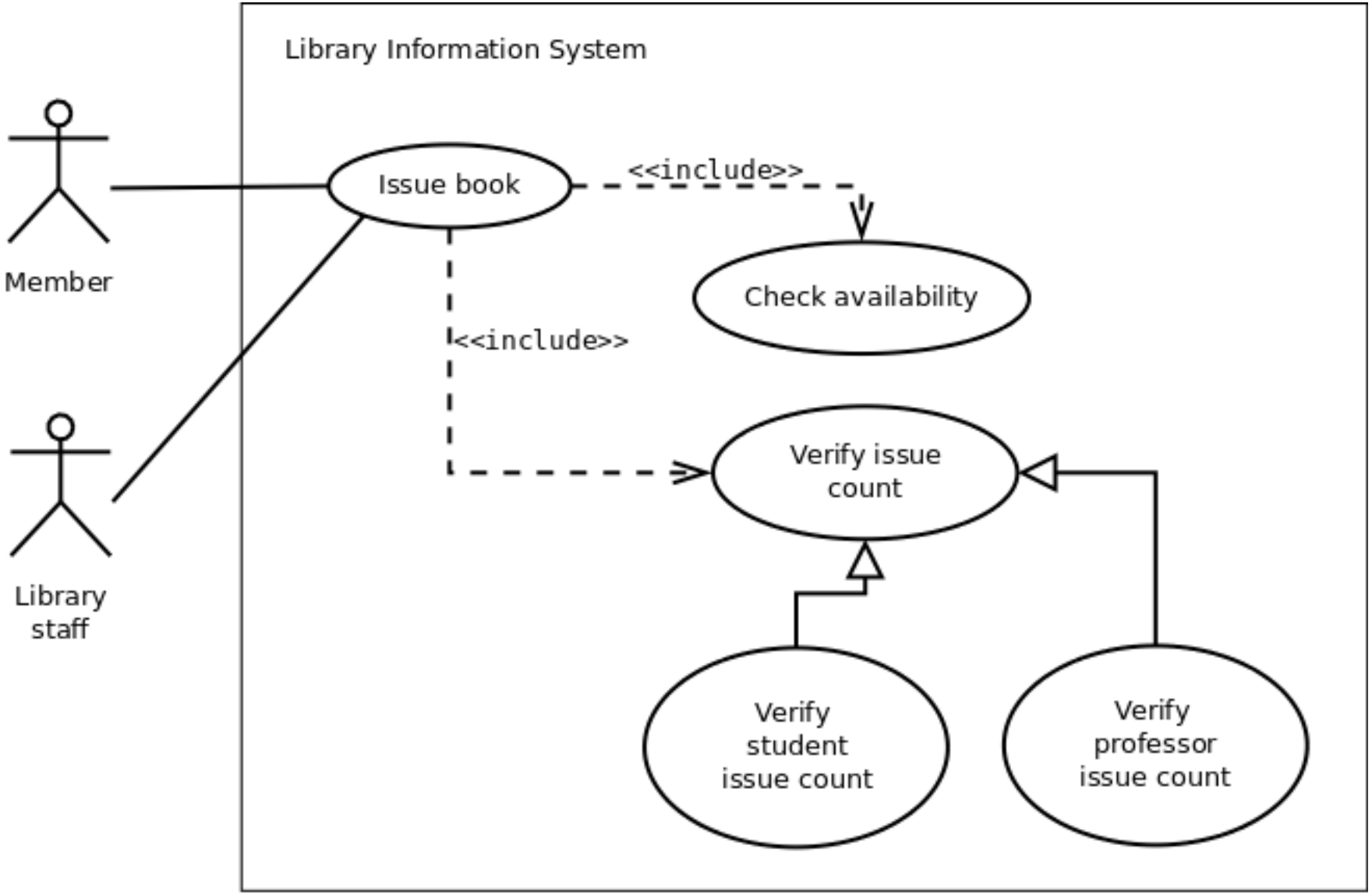
System Modelling – UML – Usecase Model

➢Example



System Modelling – UML – Usecase Model

➢Example



System Modelling – UML – Activity Model

- Activity diagrams are behavioral diagrams that visually represent the flow of control in a system.
- They are like flowcharts but can represent concurrent activities.
- They can't depict message passing among objects and thus can't be directly translated into code.
- Activity diagrams are suitable for confirming logic with business users, especially when the business logic is complex.

System Modelling – UML – Activity Model

- **Components:**
- Activity: Represents an action (rounded rectangle). Special types: initial (filled circle) and final (filled circle with a border) nodes.
- Flow: Directed arrow showing the transfer of control. May include guard conditions.
- Decision: Diamond shape with one incoming flow and multiple outgoing flows, based on conditions.
- Merge: Diamond shape with multiple incoming flows and one outgoing flow; processing continues when at least one control reaches the node.

System Modelling – UML – Activity Model

- Components:
- Fork: Black bar with one incoming flow and multiple outgoing flows, indicating parallel activities.
- Join: Black bar with multiple incoming flows and one outgoing flow, synchronizing concurrent activities (all incoming controls must be completed).
- Note: Used to attach textual information (comments, constraints) to components.
- Partition (Swimlane): Logically groups components into areas, often corresponding to organizational units or actors (not mandatory).

System Modelling – UML – Activity Model

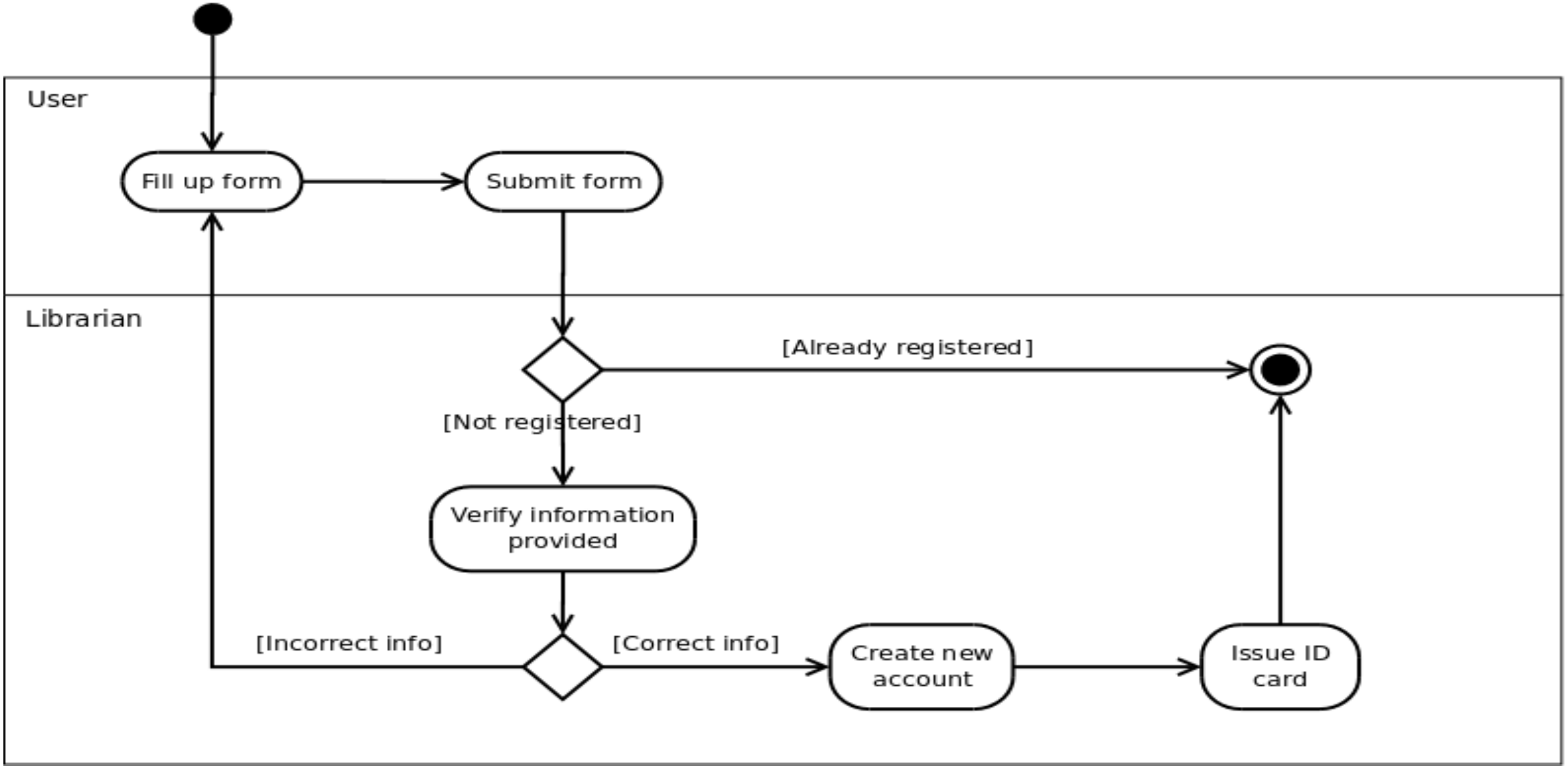
Activity	An Activity
Flow	[A Flow]
Decision	
Merge	

Fork	
Join	
Note	A simple note

System Modelling – UML – Activity Model

- Guidelines
  - Identify tiny pieces of work being performed by the system
  - Identify the next logical activity that should be performed
  - Think about all those conditions that should be made, and all those constraints that should be satisfied, before one can move to the next activity
  - Put non-trivial guard conditions on the edges to avoid confusion

System Modelling – UML – Activity Model - Example



System Modelling – UML – State Chart Model

- A statechart diagram is a visual representation of a system, showing its states and the events that cause transitions between them.
- It models the behavior of a system with well-defined states.
- Statechart diagrams are typically used to model complex systems.

System Modelling – UML – State Chart Model

- **Components**
  - State: A distinct stage an object/system passes through.
  - Initial: The state an object is in when created (represented by a filled black circle).
  - Final: A state from which the object does not move to another state (represented by an unfilled circle with an inner black-filled circle);
  - Intermediate: Any state that is not initial or final (represented by a rounded rectangle).

System Modelling – UML – State Chart Model

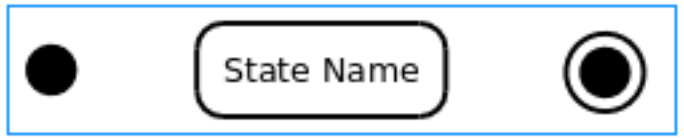
- **Components**
  - Name compartment: Contains the name of the state.
  - Internal transitions compartment: Lists internal activities performed while in the state, using the syntax: action-label / action-expression.
  - Special action labels include:
    - Entry: Activity performed upon entering the state.
    - Exit: Activity performed upon exiting the state.
    - Do: Activity performed while in the state.
    - Include: Invocation of a sub-machine.

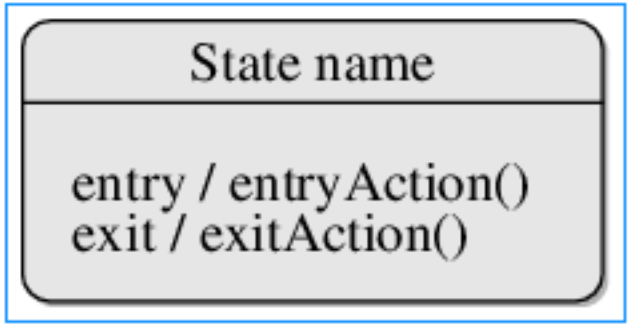
System Modelling – UML – State Chart Model

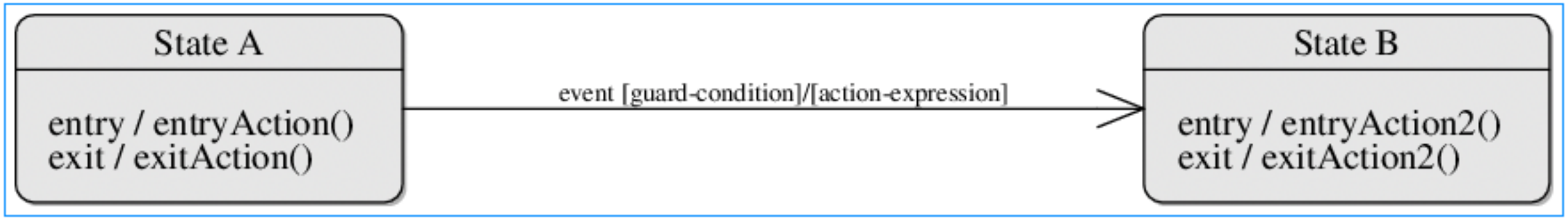
- **Components**
  - Transition: Movement from one state to another in response to an external stimulus or internal event (represented by a solid arrow). Labeled as: event [guard-condition] / [action-expression].
  - Event: What causes the transition (mandatory, written in past tense).
  - Guard-condition: Precondition(s) that must be true for the transition to happen (optional).
  - Action-expression: Action(s) performed as a result of the transition (optional).

System Modelling – UML – State Chart Model

➢ Components







System Modelling – UML – State Chart Model

- **Guidelines**
  - Identify the distinct states the system passes through.
  - Identify the events (and preconditions) that cause state transitions.
  - Identify what activities are performed while the system remains in a given state.

System Modelling – UML – Structural Aspect - Sequence

- **Sequence Diagram**: Shows object interactions and message passing over time.

- **Elements**: Objects, lifelines, and messages.

- **Object**: Instance of a class. Represented by a box at the top with object name : class name (underlined).

- **Lifeline**: Vertical line representing an object's existence.

- Activation box on lifeline shows when the object is active.

System Modelling – UML – Structural Aspect - Sequence

- **Messages**: Communication between objects. Arrows labeled with message name.
  - **Synchronous**: Sender waits for receiver to finish. Filled arrowhead.
  - **Asynchronous**: Sender doesn't wait. Open arrowhead.
  - **Return**: Value returned from a message. Dashed arrow with open arrowhead.
  - **Self-Message**: Object sending a message to itself.

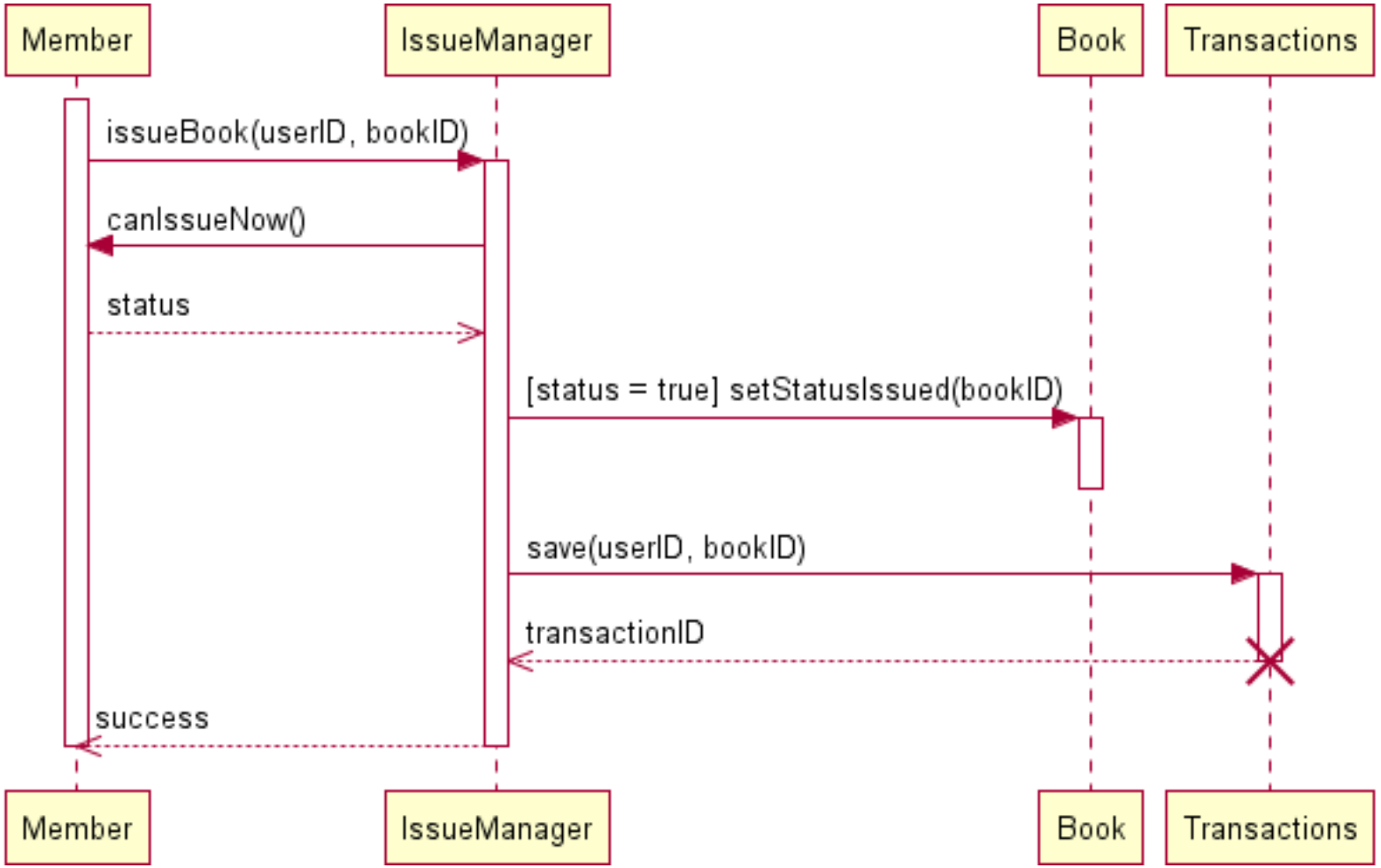
System Modelling – UML – Structural Aspect - Sequence

➢ **Messages**:

Message Type	Notation
Synchronous message	
Asynchronous message	
Response message	

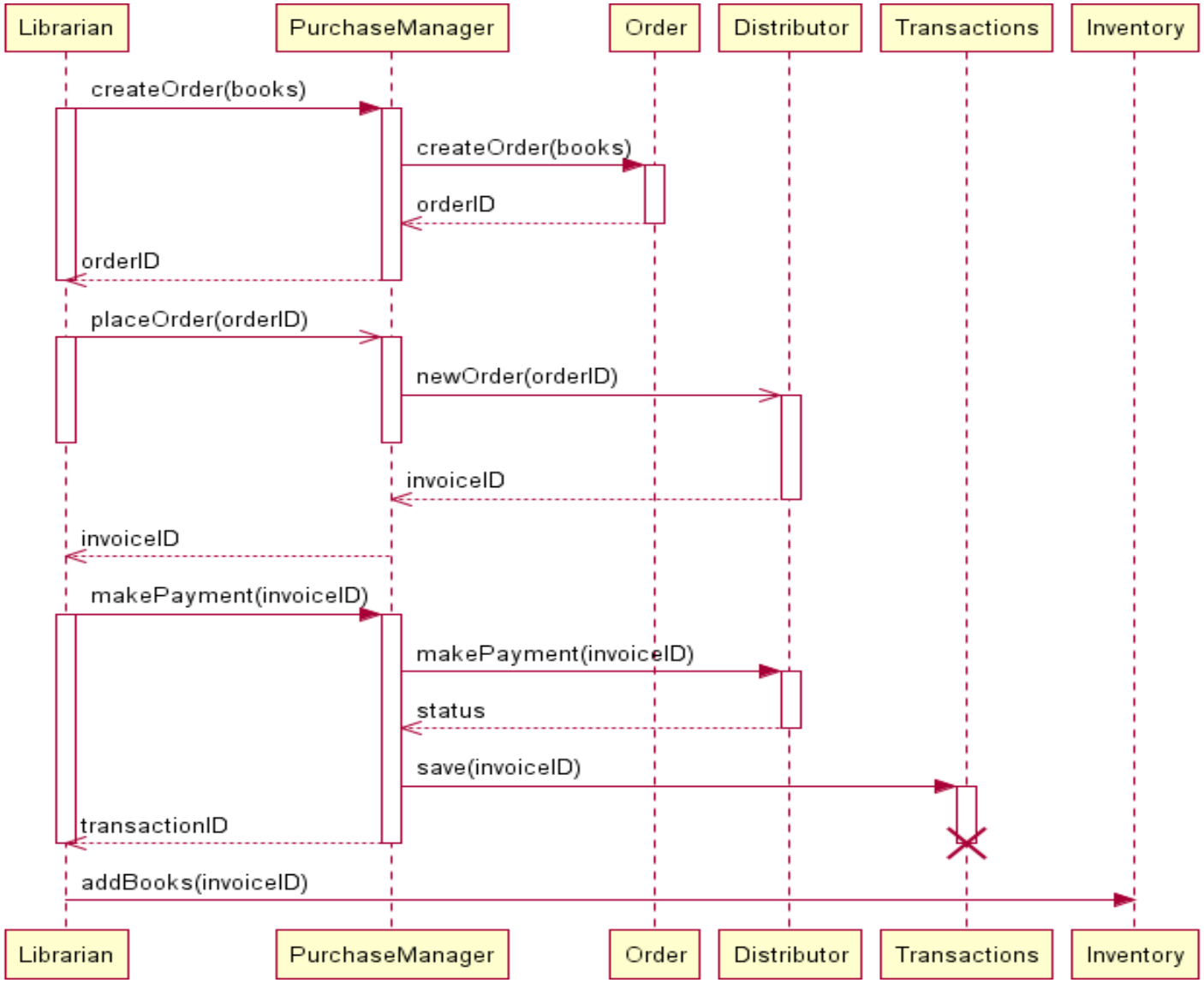
System Modelling – UML – Structural Aspect - Sequence

➢ **Example Issue Book**:



System Modelling – UML – Structural Aspect - Sequence

➢ **Example Purchase Book**:



System Modelling – UML – Structural Aspect - Class

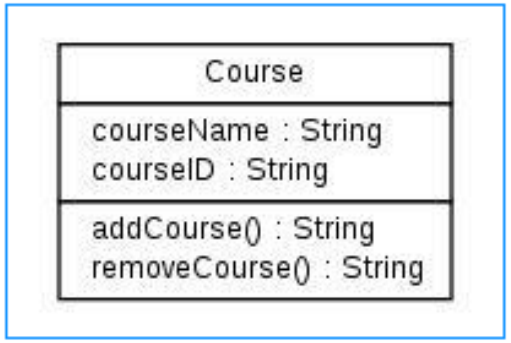
- Class Diagram: Graphical representation of a system's static structure.
- Elements: Classes, attributes, operations, and relationships.
- Class: Blueprint for objects with similar data and behavior. Represented by a rectangle with three compartments:
  - **Class Name**: Unique identifier.
  - **Attributes:** Properties of the class (data members).
  - **Operations:** Actions the class can perform (member functions).

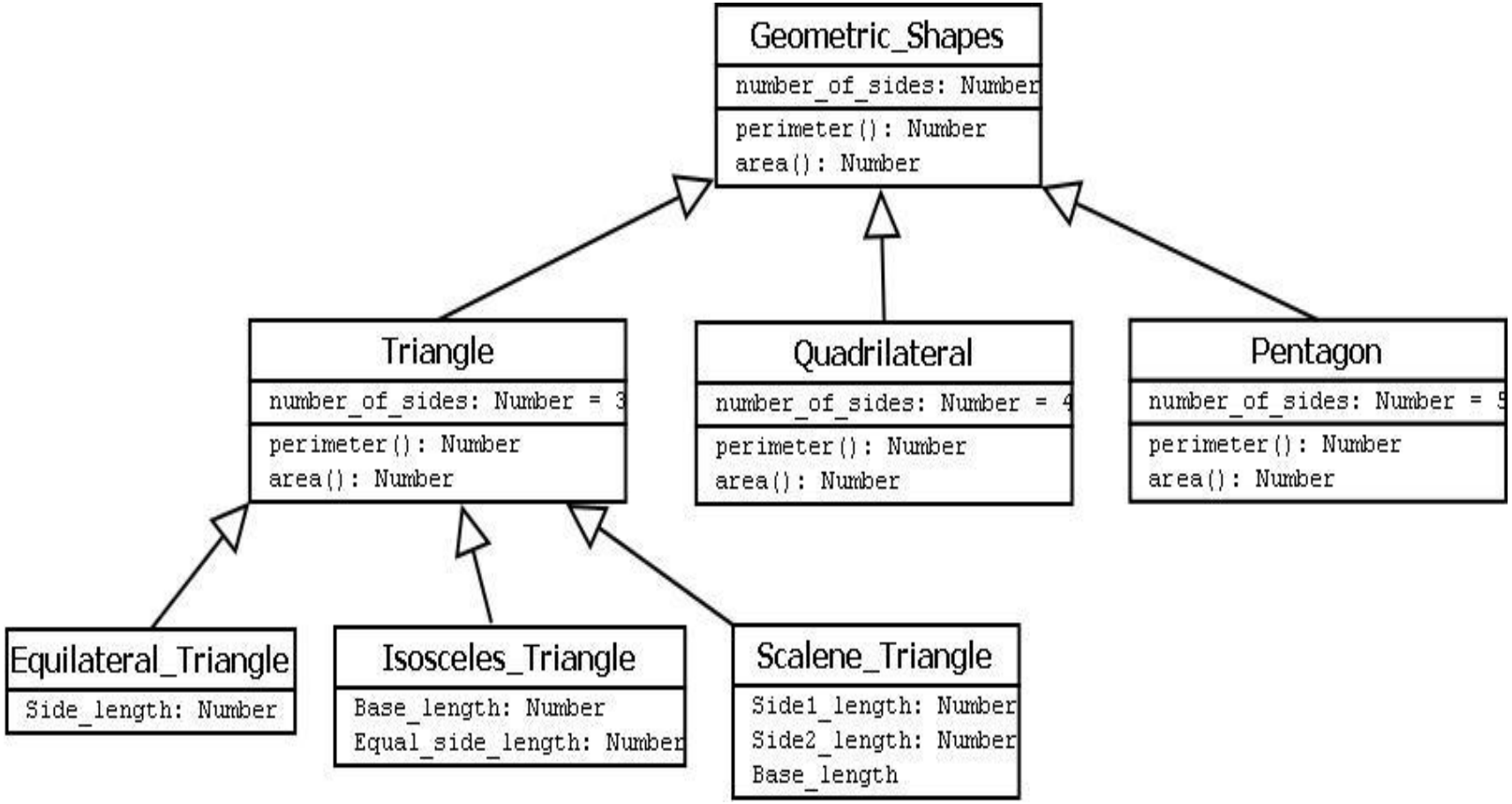
System Modelling – UML – Structural Aspect - Class

- **Generalization/Specialization:** Describes inheritance relationships. Derived classes inherit properties from parent classes. "ISA" relationship.

- **Relationships: Connections between classes.**
  - **Association:** Instance-level connection allowing message exchange. Line connecting classes. Can have name, role names, and multiplicity. Can be unidirectional.
  - **Aggregation:** "Part-whole" relationship where the whole contains the parts. Weaker ownership than composition. Represented by a hollow diamond on the whole side.
  - **Composition:** Strong "part-whole" relationship where the whole owns the parts. Part lifecycle depends on the whole. Represented by a filled diamond on the whole side.
  - **Multiplicity:** Specifies the number of instances of one class related to another. (e.g., 1, 0..1, , 1.., n).

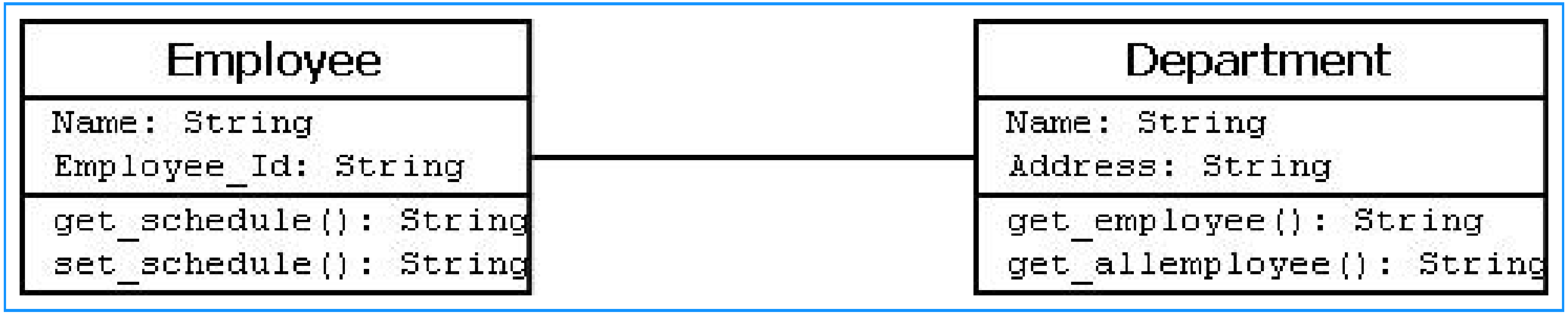
System Modelling – UML – Structural Aspect - Class





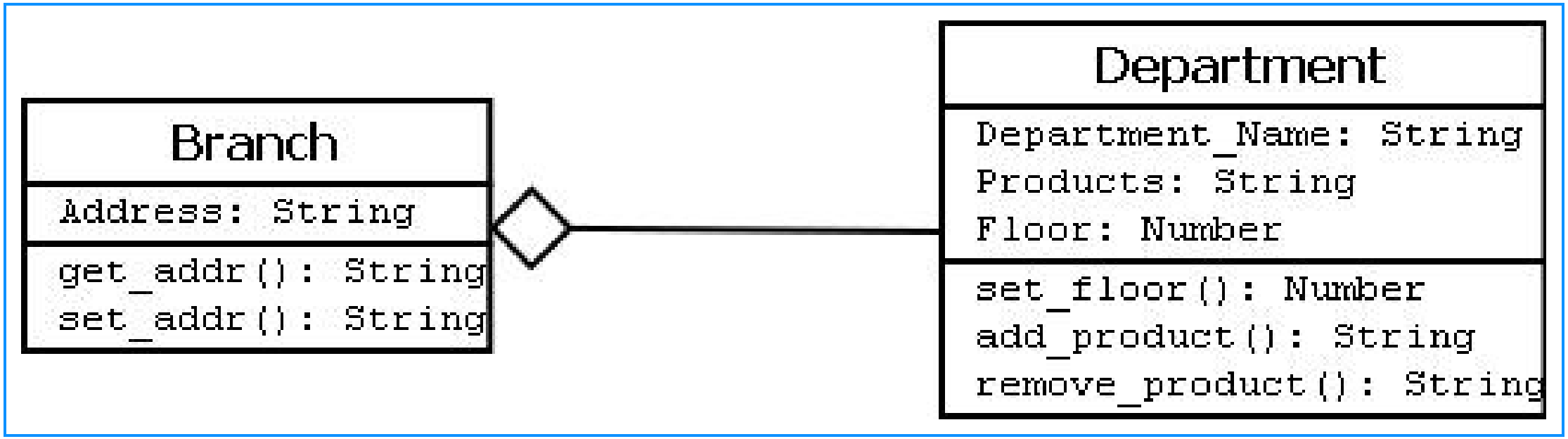
System Modelling – UML – Structural Aspect - Class

- Association Example



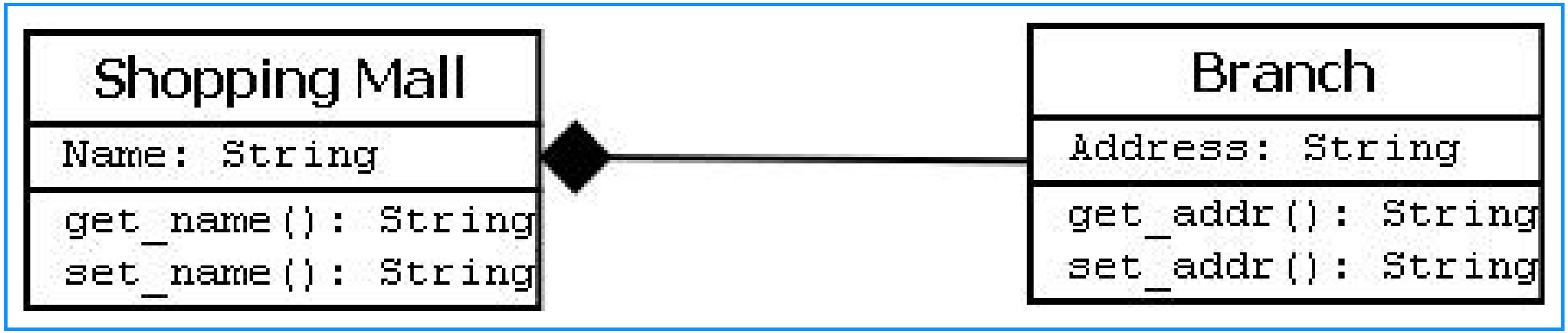
System Modelling – UML – Structural Aspect - Class

- Aggregation Example



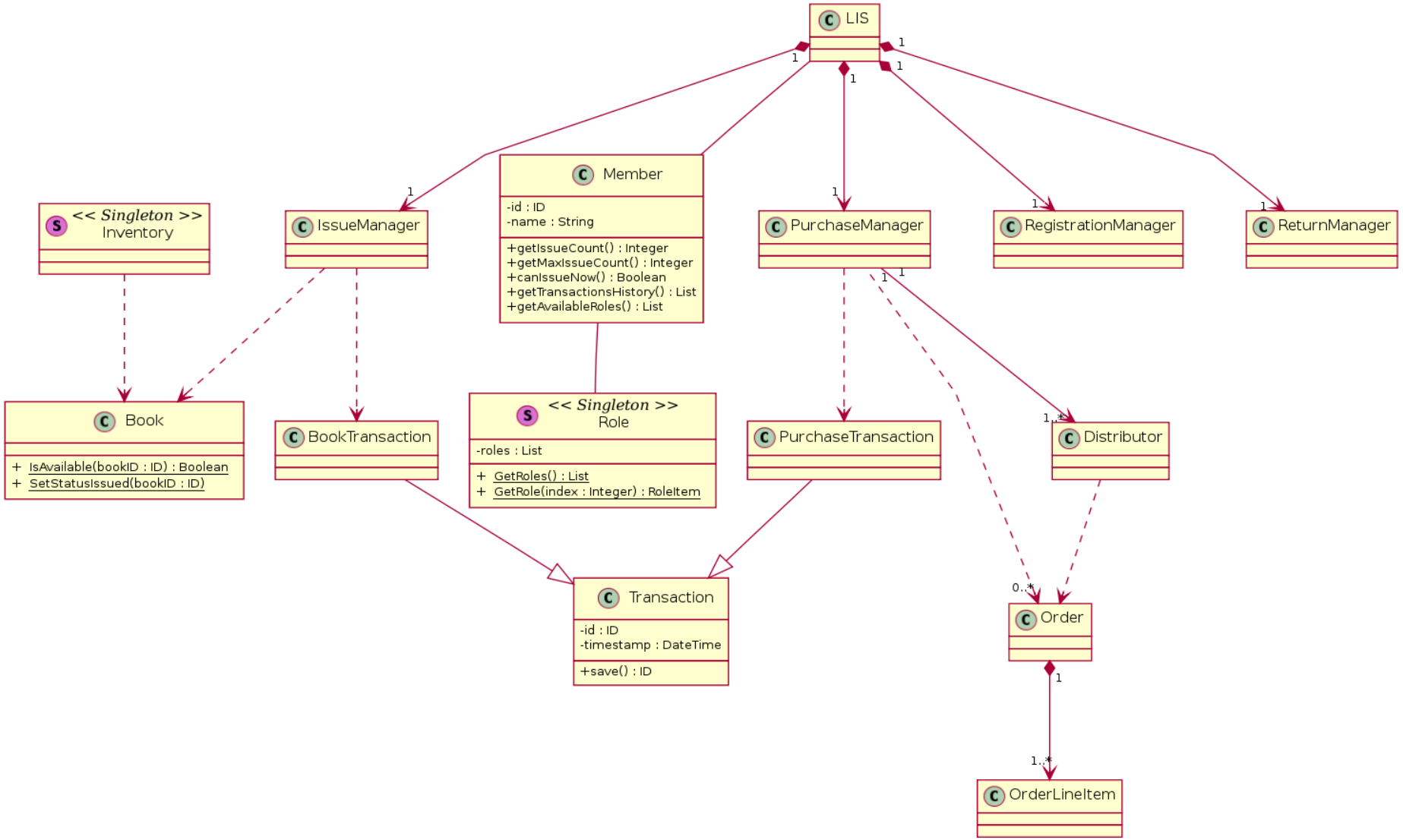
System Modelling – UML – Structural Aspect - Class

- Composition Example



System Modelling – UML – Structural Aspect - Class

➢Example



System Modelling – UML – Component Model

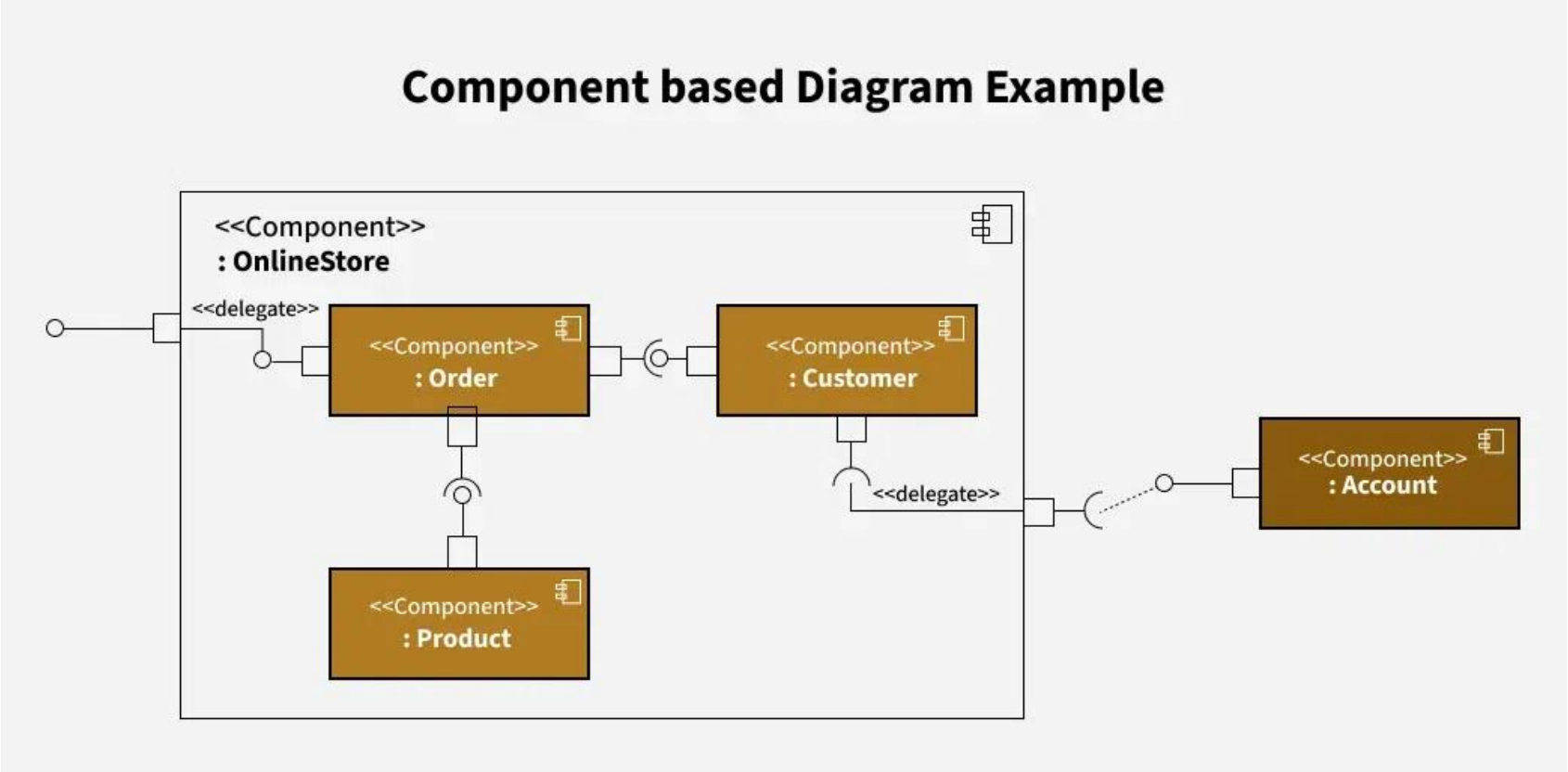
- Model breaks down a large system into smaller, manageable components, modeling the physical parts of the system like files, executables, and libraries.
- It provides a visual representation of the components and their relationships, helping to form an executable system.
- Each component is a single, replaceable unit with hidden implementation details, communicating through interfaces

System Modelling – UML – Component Model

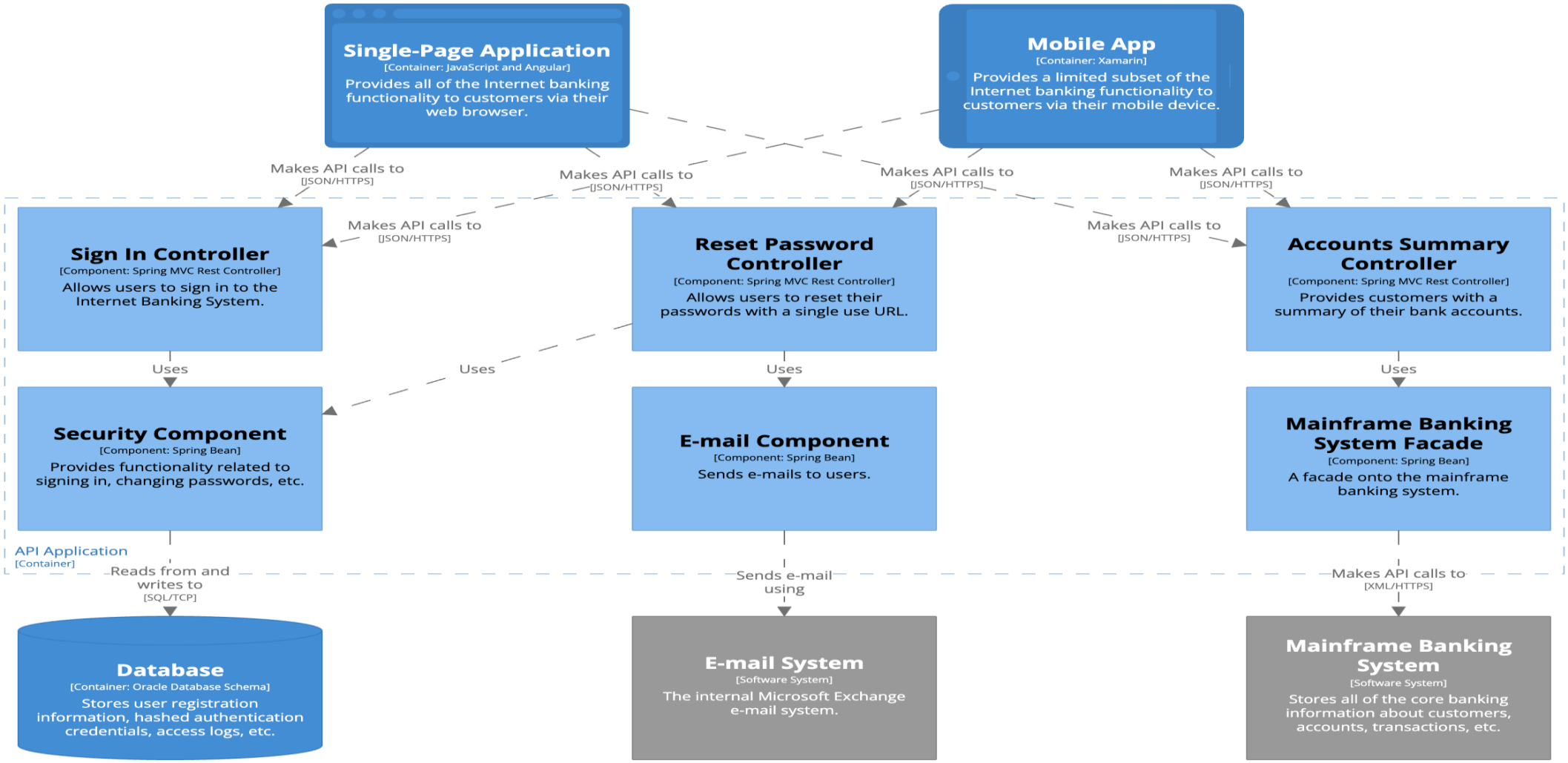
- To depict the functionality and behavior of components within a system at a specific time
- Components: Represented as rectangles with the <<component>> stereotype or a component icon. They can be basic building blocks or larger subsystems.
- Interfaces: Define how components interact with each other, with components providing and requiring interfaces

System Modelling – UML – Component Model

- Connectors: Illustrate the relationships between components4. Assembly connectors link provided interfaces to required interfaces



System Modelling – UML – Component Model



[Component] Internet Banking System - API Application

The component diagram for the API Application - diagram created with Structurizr.
Saturday, 11 November 2023 at 09:04 Greenwich Mean Time

System Modelling – UML – Deployment Model

- Visualizes the physical hardware and software deployment in a system.
- Shows how software components are deployed on hardware.
- Depicts the relationships between software and hardware components

System Modelling – UML – Deployment Model

- Key Elements
  - **Nodes:** Represent hardware devices (e.g., servers, PCs, mobile phones) or software execution environments (e.g., operating systems, JVMs). They are the most important part of the diagram.
  - **Artifacts:** Represent concrete software elements like executable files, libraries, configuration files, etc.
  - Communication Paths: Solid lines that show the communication links between nodes. These can be labeled with communication protocols (e.g., HTTP, TCP/IP)7.

System Modelling – UML – Deployment Model

- Nodes
  - **Device Nodes**: Computing resources with processing capabilities4.
  - **Execution Environment Nodes (EEN)**: Computer systems residing within a device node, like operating systems or JVMs.
  - Nodes can be nested to show more detail, like a disk inside a PC.
  - Node instances can be shown to represent specific instances

System Modelling – UML – Deployment Model

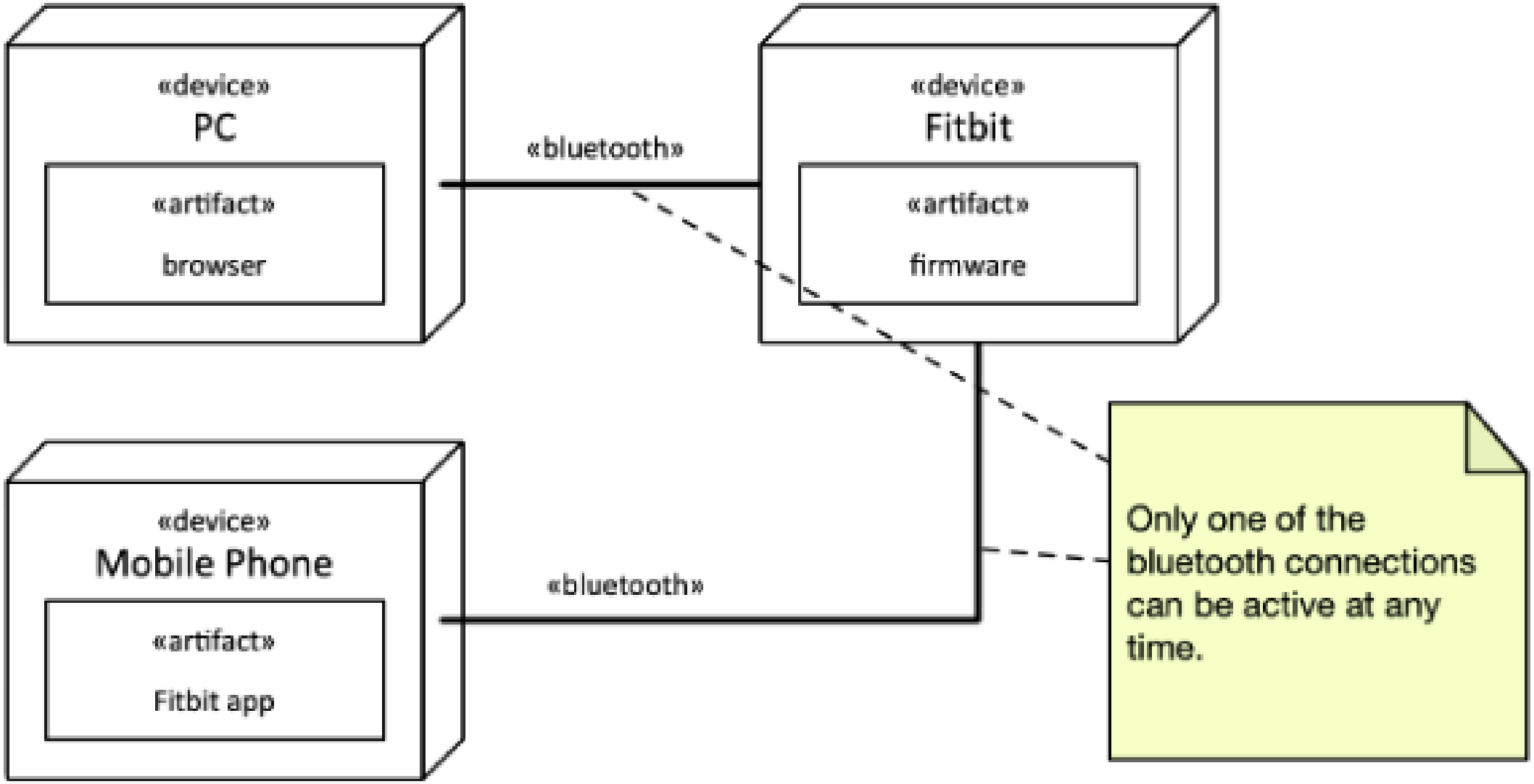
- Artifacts in Detail:
  - Represent software that runs on hardware.
  - Examples: databases, configuration files, executable files

- Relationships:
  - Association: Represents a structural connection.
  - Dependency: Indicates that one element depends on another.
  - Deployment: Shows how artifacts are deployed to nodes. Artifacts can be deployed to nodes by nesting or using a dashed line arrow labeled "deploy"

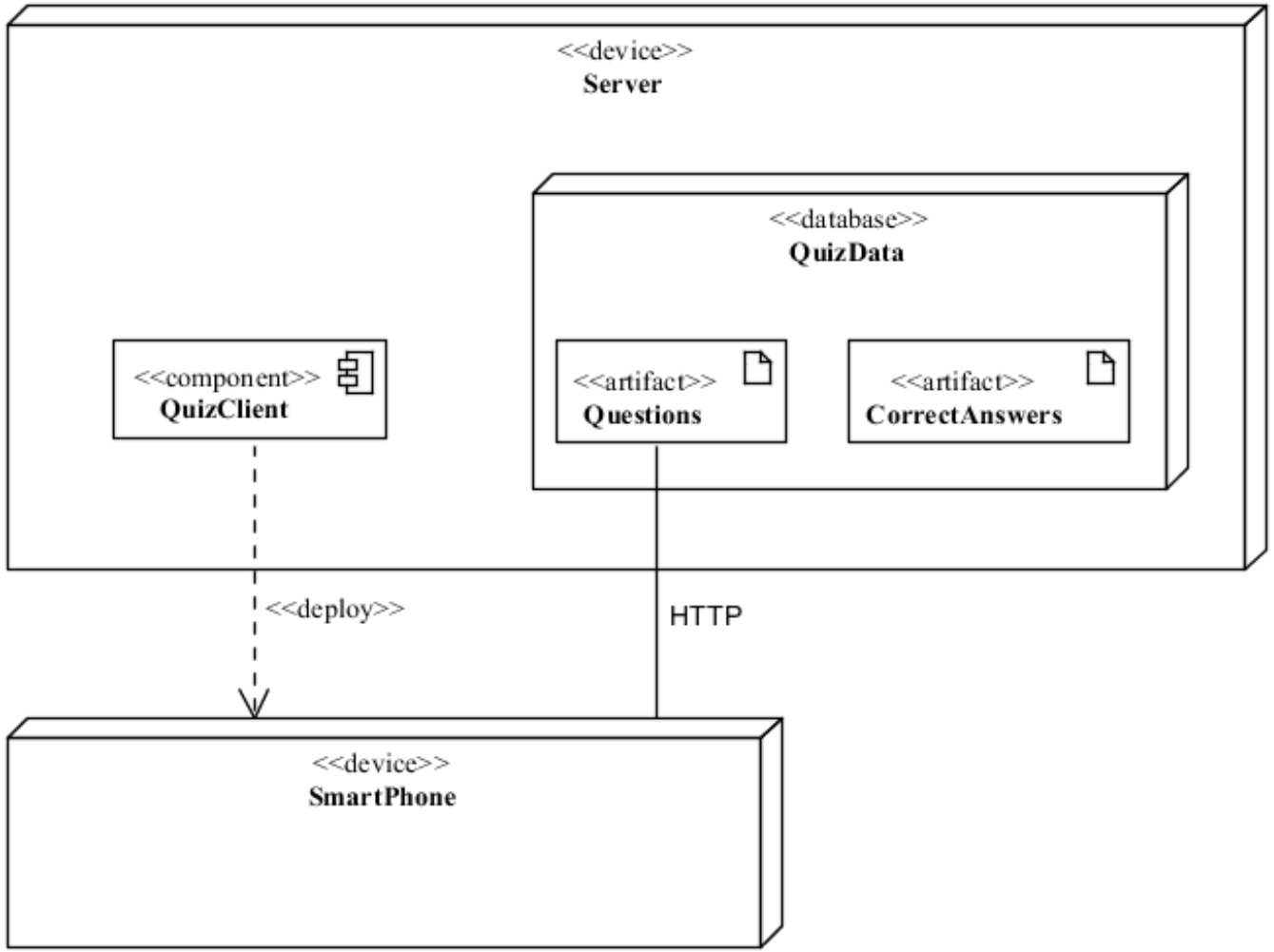
System Modelling – UML – Deployment Model

- Guidelines
  - Identify the nodes (hardware and software)
  - Outline the software artifacts
  - Establish relationships between nodes
  - Configure artifacts on nodes
  - Detail the environment (e.g., OS, cloud platform)
  - Annotate the diagram with notes and constraints.

System Modelling – UML – Deployment Model



System Modelling – UML – Deployment Model



System Modelling – UML – Summary

- Class Diagram: It visualizes a system's classes, their properties, and the relationships between them. It is the foundation for component and deployment diagrams
- Component Diagram: It verifies that the required functionalities are implemented by components and is a communication tool to discuss and improve system design. It breaks down complex systems into smaller components and visualizes the relationships between these components.

System Modelling – UML – Summary

- Use Case Diagram: Used during the analysis phase of a project to identify system functionality by describing the interaction of people or external devices with the system. It provides a high-level overview of the relationships between actors and systems, making it suitable for explaining a system to a non-technical audience
- Sequence Diagram: Describes how operations are carried out. It depicts the chronologically structured event flow through a use case and presents the communication relationships between objects

System Modelling – UML – Summary

- Communication Diagram: Visualizes the relationship between objects collaborating to perform a particular task and models the logic of the implementation for a complex operation. It focuses on messages passed between objects and can represent the same information as a sequence diagram using different objects
- Deployment Diagram: Describes the hardware components used in system implementations, the execution environments, and the artifacts deployed on the hardware1. It helps visualize the hardware topology system, model physical hardware elements, and the communication relationships between them, as well as plan the architecture of the system.

System Modelling – UML – Summary

- State Diagram: Models the dynamic nature of a system by describing all possible states of an object as events occur, from creation to termination.
- Activity Diagram: Graphically represents business or operational workflows to show the activity of any part or component in the system.

System Modelling – UML – Summary

- Object Diagram: Shows the relationship between objects using real-world examples and illustrates how a system will look at any given time